



UNIVERSIDADE FEDERAL DE ALAGOAS

CAMPUS A.C. SIMÕES

IC - UFAL

LUCAS TRINDADE DE SOUZA

LISTA 1 DE INTELIGÊNCIA ARTIFICIAL

MACEIÓ/AL

2026

LUCAS TRINDADE DE SOUZA

LISTA 1 DE INTELIGÊNCIA ARTIFICIAL

Trabalho solicitado pelo professor
Evandro de Barros Costa, na matéria de
Inteligência Artificial (ECOM 031),
para compor a nota AB1.

MACEIÓ/AL

1. Questão 1:

1.1 (i) Aumento da base de dados

Para resolver o item (i) da primeira questão, foi necessário expandir a base de dados original que continha 4 atributos preditivos e 14 exemplos. A ampliação consistiu na adição de 2 novos atributos ("Emprego" e "Idade") para totalizar 6 atributos preditivos, e na criação de 16 novos exemplos (E15 a E30) distribuídos entre as classes de Risco (Baixo, Alto e Moderado).

Dos novos atributos, idade é classificada como “Jovem e Adulto” e emprego é dividido em “Fixo, Autônomo, Desempregado”

A	B	C	D	E	F	G	H
1 Exemplo	História de Crédito	Dívida	Garantia	Renda	Emprego	Idade	Risco
2 E1	Ruim	Alta	Nenhuma	\$0 a \$15k	Fixo	Adulto	Alto
3 E2	Desconhecida	Alta	Nenhuma	\$15 a \$35k	Autônomo	Jovem	Alto
4 E3	Desconhecida	Baixa	Nenhuma	\$15 a \$35k	Fixo	Jovem	Moderado
5 E4	Desconhecida	Baixa	Nenhuma	\$0 a \$15k	Desempregado	Adulto	Alto
6 E5	Desconhecida	Baixa	Nenhuma	Acima de \$35k	Fixo	Adulto	Baixo
7 E6	Desconhecida	Baixa	Adequada	Acima de \$35k	Fixo	Adulto	Baixo
8 E7	Ruim	Baixa	Nenhuma	\$0 a \$15k	Autônomo	Jovem	Alto
9 E8	Ruim	Baixa	Adequada	Acima de \$35k	Fixo	Adulto	Moderado
10 E9	Boa	Baixa	Nenhuma	Acima de \$35k	Fixo	Jovem	Baixo
11 E10	Boa	Alta	Adequada	Acima de \$35k	Fixo	Adulto	Baixo
12 E11	Boa	Alta	Nenhuma	\$0 a \$15k	Autônomo	Jovem	Alto
13 E12	Boa	Alta	Nenhuma	\$15 a \$35k	Fixo	Jovem	Moderado
14 E13	Boa	Alta	Nenhuma	Acima de \$35k	Fixo	Adulto	Baixo
15 E14	Ruim	Alta	Nenhuma	\$15 a \$35k	Desempregado	Adulto	Alto
16 E15	Boa	Baixa	Adequada	\$15 a \$35k	Fixo	Adulto	Baixo
17 E16	Ruim	Alta	Nenhuma	\$0 a \$15k	Desempregado	Jovem	Alto
18 E17	Desconhecida	Baixa	Adequada	\$15 a \$35k	Autônomo	Adulto	Moderado
19 E18	Boa	Baixa	Nenhuma	Acima de \$35k	Fixo	Adulto	Baixo
20 E19	Ruim	Alta	Adequada	\$15 a \$35k	Fixo	Adulto	Moderado
21 E20	Desconhecida	Alta	Nenhuma	\$0 a \$15k	Desempregado	Adulto	Alto
22 E21	Boa	Alta	Adequada	Acima de \$35k	Autônomo	Adulto	Baixo
23 E22	Ruim	Baixa	Nenhuma	\$15 a \$35k	Fixo	Jovem	Moderado
24 E23	Desconhecida	Baixa	Nenhuma	Acima de \$35k	Fixo	Adulto	Baixo
25 E24	Ruim	Alta	Nenhuma	\$0 a \$15k	Autônomo	Jovem	Alto
26 E25	Boa	Baixa	Adequada	\$0 a \$15k	Fixo	Adulto	Moderado
27 E26	Desconhecida	Alta	Adequada	Acima de \$35k	Fixo	Jovem	Baixo
28 E27	Boa	Baixa	Nenhuma	\$15 a \$35k	Autônomo	Jovem	Moderado
29 E28	Ruim	Alta	Nenhuma	Acima de \$35k	Desempregado	Adulto	Alto
30 E29	Desconhecida	Baixa	Adequada	\$0 a \$15k	Fixo	Adulto	Moderado
31 E30	Boa	Alta	Nenhuma	\$0 a \$15k	Fixo	Jovem	Alto

1.2 (ii) Construção manual das árvores

1.2.1 Algoritmo ID3:

O algoritmo ID3 utiliza o Ganho de Informação baseado na Entropia de Shannon para escolher os nós. A fórmula da Entropia é $H(S) = - \sum p_i \log_2(p_i)$. O ID3 cria ramificações múltiplas para cada valor categórico do atributo escolhido.

Ao particionar o conjunto S pelos valores possíveis v de um determinado atributo N, a entropia esperada da partição é calculada pela média ponderada das entropias dos subconjuntos S_v : $H(S,N) = \sum_{v \in \text{Valores}(N)} |S_v|/|S| H(S_v)$

$$v \in \text{Valores}(N)$$

O Ganho de Informação $IG(S,N)$ mede a redução esperada na entropia obtida ao segmentar S segundo o atributo N: $IG(S,N) = H(S) - H(S,N)$

Ao fazer os cálculos, encontraremos qual deve ser a raiz da nossa árvore.

Entropia Inicial (Conjunto S0)

$$H(S_0) = - \left(\frac{11}{30}\right) \log_2\left(\frac{11}{30}\right) - \left(\frac{9}{30}\right) \log_2\left(\frac{9}{30}\right) - \left(\frac{10}{30}\right) \log_2\left(\frac{10}{30}\right)$$
$$H(S_0) \approx 0,5307 + 0,5211 + 0,5283 \approx 1,5801$$

1.2.1.1 Cálculos de Entropia e Ganho de Informação por Atributo

HISTÓRIA DE CRÉDITO

- **Boa** (11 ex: 2A, 3M, 6B): $H(S_{Boa}) = - \left(\frac{2}{11}\right) \log_2\left(\frac{2}{11}\right) - \left(\frac{3}{11}\right) \log_2\left(\frac{3}{11}\right) - \left(\frac{6}{11}\right) \log_2\left(\frac{6}{11}\right) \approx 1,434$
- **Desconhecida** (10 ex: 3A, 3M, 4B): $H(S_{Desc}) = - (0,3) \log_2(0,3) - (0,3) \log_2(0,3) - (0,4) \log_2(0,4) \approx 1,571$
- **Ruim** (9 ex: 6A, 3M, 0B): $H(S_{Ruim}) = - \left(\frac{6}{9}\right) \log_2\left(\frac{6}{9}\right) - \left(\frac{3}{9}\right) \log_2\left(\frac{3}{9}\right) \approx 0,918$
- **Entropia Ponderada:** $\left(\frac{11}{30} \times 1,434\right) + \left(\frac{10}{30} \times 1,571\right) + \left(\frac{9}{30} \times 0,918\right) \approx 1,3249$
- **Ganho:** $1,5801 - 1,3249 = 0,2552$

DÍVIDA

- **Alta** (15 ex: 9A, 2M, 4B):
- $H(S_{Alta}) = - \left(\frac{9}{15}\right) \log_2\left(\frac{9}{15}\right) - \left(\frac{2}{15}\right) \log_2\left(\frac{2}{15}\right) - \left(\frac{4}{15}\right) \log_2\left(\frac{4}{15}\right) \approx 1,338$
- **Baixa** (15 ex: 2A, 7M, 6B): $H(S_{Baixa}) = - \left(\frac{2}{15}\right) \log_2\left(\frac{2}{15}\right) - \left(\frac{7}{15}\right) \log_2\left(\frac{7}{15}\right) - \left(\frac{6}{15}\right) \log_2\left(\frac{6}{15}\right) \approx 1,429$

- **Entropia Ponderada:** $(\frac{15}{30} \times 1,338) + (\frac{15}{30} \times 1,429) \approx 1,3835$
- **Ganho:** $1,5801 - 1,3835 = 0,1966$

GARANTIA

- **Nenhuma** (20 ex: 11A, 4M, 5B): $H(S_{\text{Nenhuma}}) = -(\frac{11}{20})\log_2(\frac{11}{20}) - (\frac{4}{20})\log_2(\frac{4}{20}) - (\frac{5}{20})\log_2(\frac{5}{20}) \approx 1,438$
- **Adequada** (10 ex: 0A, 5M, 5B): $H(S_{\text{Adequada}}) = -(0,5)\log_2(0,5) - (0,5)\log_2(0,5) = 1,000$
- **Entropia Ponderada:** $(\frac{20}{30} \times 1,438) + (\frac{10}{30} \times 1,000) \approx 1,2920$
- **Ganho:** $1,5801 - 1,2920 = 0,2881$

RENDIA

- **\$0 a \$15k** (10 ex: 8A, 2M, 0B): $H(S_{0-15k}) = -(0,8)\log_2(0,8) - (0,2)\log_2(0,2) \approx 0,7219$
- **\$15 a \$35k** (9 ex: 2A, 6M, 1B): $H(S_{15-35k}) = -(\frac{2}{9})\log_2(\frac{2}{9}) - (\frac{6}{9})\log_2(\frac{6}{9}) - (\frac{1}{9})\log_2(\frac{1}{9}) \approx 1,224$
- **Acima de \$35k** (11 ex: 1A, 1M, 9B): $H(S_{>35k}) = -(\frac{1}{11})\log_2(\frac{1}{11}) - (\frac{1}{11})\log_2(\frac{1}{11}) - (\frac{9}{11})\log_2(\frac{9}{11}) \approx 0,865$
- **Entropia Ponderada:** $(\frac{10}{30} \times 0,7219) + (\frac{9}{30} \times 1,224) + (\frac{11}{30} \times 0,865) \approx 0,9250$
- **Ganho:** $1,5801 - 0,9250 = 0,6551$

EMPREGO

- **Fixo** (18 ex: 2A, 7M, 9B): $H(S_{\text{Fixo}}) = -(\frac{2}{18})\log_2(\frac{2}{18}) - (\frac{7}{18})\log_2(\frac{7}{18}) - (\frac{9}{18})\log_2(\frac{9}{18}) \approx 1,382$
- **Autônomo** (7 ex: 4A, 2M, 1B): $H(S_{\text{Autônomo}}) = -(\frac{4}{7})\log_2(\frac{4}{7}) - (\frac{2}{7})\log_2(\frac{2}{7}) - (\frac{1}{7})\log_2(\frac{1}{7}) \approx 1,378$
- **Desempregado** (5 ex: 5A, 0M, 0B): $H(S_{\text{Desempregado}}) = -(1)\log_2(1) = 0,000$
- **Entropia Ponderada:** $(\frac{18}{30} \times 1,382) + (\frac{7}{30} \times 1,378) + (\frac{5}{30} \times 0) \approx 1,1507$
- **Ganho:** $1,5801 - 1,1507 = 0,4294$

IDADE

- **Adulto** (18 ex: 5A, 5M, 8B): $H(S_{\text{Adulto}}) = -(\frac{5}{18})\log_2(\frac{5}{18}) - (\frac{5}{18})\log_2(\frac{5}{18}) - (\frac{8}{18})\log_2(\frac{8}{18}) \approx 1,547$
- **Jovem** (12 ex: 6A, 4M, 2B): $H(S_{\text{Jovem}}) = -(\frac{6}{12})\log_2(\frac{6}{12}) - (\frac{4}{12})\log_2(\frac{4}{12}) - (\frac{2}{12})\log_2(\frac{2}{12}) \approx 1,459$
- **Entropia Ponderada:** $(\frac{18}{30} \times 1,547) + (\frac{12}{30} \times 1,459) \approx 1,5116$
- **Ganho:** $1,5801 - 1,5116 = 0,0685$

1.2.1.2. Conclusão da Raiz

Comparando todos os ganhos de informação gerados a partir da base em:

- **Renda:** 0,6551
- **Emprego:** 0,4294
- **Garantia:** 0,2881
- **História de Crédito:** 0,2552
- **Dívida:** 0,1966
- **Idade:** 0,0685

O atributo **Renda** possui o maior Ganho de Informação e, portanto, será o nó raiz da árvore de decisão gerada pelo algoritmo ID3.

A partir do nó raiz **Renda**, a base de dados de 30 exemplos divide-se em três subconjuntos. Para prosseguir com o algoritmo ID3, calculamos a entropia e o ganho de informação para cada ramo subsequente, avaliando os atributos restantes.

Ramo 1: Renda = "\$0 a \$15k"

Este subconjunto contém 10 exemplos: E1, E4, E7, E11, E16, E20, E24, E25, E29, E30.

- **Distribuição das classes:** 8 Alto, 2 Moderado, 0 Baixo.
- **Entropia Inicial:** $H(S_{0-15k}) = - (0,8)\log_2(0,8) - (0,2)\log_2(0,2) \approx 0,7219$

Ao analisar os atributos para estes 10 exemplos, o atributo **Garantia** fornece uma separação perfeita:

- **Nenhuma** (8 exemplos: E1, E4, E7, E11, E16, E20, E24, E30): Todos são da classe **Alto**. $H(\text{Nenhuma}) = 0$
- **Adequada** (2 exemplos: E25, E29): Todos são da classe **Moderado**. $H(\text{Adequada}) = 0$
- **Ganho de Informação (Garantia):** $0,7219 - 0 = 0,7219$ (Ganho máximo possível. Este nó torna-se folha).

Ramo 2: Renda = "\$15k a \$35k"

Este subconjunto contém 9 exemplos: E2, E3, E12, E14, E15, E17, E19, E22, E27.

- **Distribuição das classes:** 2 Alto, 6 Moderado, 1 Baixo.
- **Entropia Inicial:** $H(S_{15-35k}) = - (\frac{2}{9})\log_2(\frac{2}{9}) - (\frac{6}{9})\log_2(\frac{6}{9}) - (\frac{1}{9})\log_2(\frac{1}{9}) \approx 1,224$

Testando os atributos restantes, o **Emprego** apresenta o maior ganho de informação:

- **Fixo** (5 ex: 0 Alto, 4 Mod, 1 Baixo): $H(\text{Fixo}) \approx 0,722$
- **Autônomo** (3 ex: 1 Alto, 2 Mod, 0 Baixo): $H(\text{Aut}) \approx 0,918$
- **Desempregado** (1 ex: 1 Alto): $H(\text{Desemp}) = 0$ (Folha: Alto)
- **Ganho (Emprego):** $1,224 - [(\frac{5}{9} \times 0,722) + (\frac{3}{9} \times 0,918)] \approx 0,517$

Como os nós "Fixo" e "Autônomo" não são puros, exigem novas ramificações:

Sub-ramo Emprego = Autônomo (3 ex: E2, E17, E27): A entropia é 0,918. Escolhe-se o atributo Dívida para desempatar, pois separa perfeitamente:

- Dívida Alta → Alto (E2)

- Dívida Baixa → Moderado (E17, E27)

Sub-ramo Emprego = Fixo (5 ex: E3, E12, E15, E19, E22; 4 Moderado e 1 Baixo, $H \approx 0,722$). Calculando o ganho dos atributos restantes:

- **Idade:** Jovem (E3, E12, E22: todos Moderado, $H = 0$) e Adulto (E15 Baixo, E19 Moderado, $H = 1$). Ganho = $0,722 - (2/5 \times 1) \approx 0,322$.
- **História de Crédito:** Boa (E12, E15, $H = 1$), Desconhecida (E3, $H = 0$) e Ruim (E19, E22, $H = 0$). Ganho $\approx 0,322$.
- **Garantia:** Nenhuma (E3, E12, E22, $H = 0$) e Adequada (E15, E19, $H = 1$). Ganho $\approx 0,322$.
- **Dívida:** ganho $\approx 0,171$.

Idade, História de Crédito e Garantia empatam em 0,322, e nenhuma separa o subconjunto por completo. Escolhe-se Idade no desempate.

- Idade = Jovem → Moderado (E3, E12, E22).
- Idade = Adulto (E15, E19) ainda é impuro. História de Crédito o separa: Boa → Baixo (E15) e Ruim → Moderado (E19).

Ramo 3: Renda = "Acima de \$35k"

Este subconjunto contém 11 exemplos: E5, E6, E8, E9, E10, E13, E18, E21, E23, E26, E28.

- **Distribuição das classes:** 1 Alto, 1 Moderado, 9 Baixo.
- **Entropia Inicial:**

$$H(S_{>35k}) = - \left(\frac{1}{11}\right)\log_2\left(\frac{1}{11}\right) - \left(\frac{1}{11}\right)\log_2\left(\frac{1}{11}\right) - \left(\frac{9}{11}\right)\log_2\left(\frac{9}{11}\right) \approx 0,865$$

Ao calcular a entropia ponderada, a **História de Crédito** oferece o melhor ganho de informação:

- **Boa** (5 ex: E9, E10, E13, E18, E21): 5 Baixo. $H(Boa) = 0$
- **Desconhecida** (4 ex: E5, E6, E23, E26): 4 Baixo. $H(Desc) = 0$
- **Ruim** (2 ex: E8, E28): 1 Alto, 1 Moderado. $H(Ruim) = 1,0$
- **Ganho (Hist. Crédito):** $0,865 - \left(\frac{2}{11} \times 1,0\right) \approx 0,683$

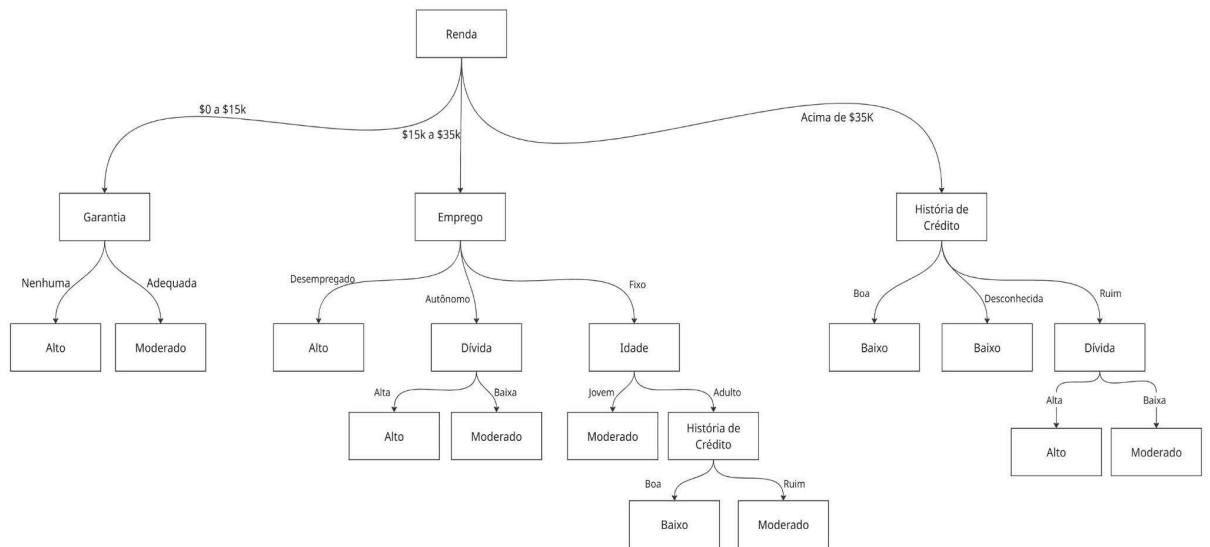
O nó "História = Ruim" (E8, E28) precisa de ser subdividido pois a entropia é máxima (1,0).

Utilizando o atributo **Dívida** (ou Emprego/Garantia, que também separariam a 100%), obtemos nós puros:

- Dívida Alta → Alto (E28)
- Dívida Baixa → Moderado (E8)

Unindo todos os cálculos efetuados sobre a tabela da imagem, a estrutura completa da árvore ID3 é:

ÁRVORE ID3



- **Renda**

- **\$0 a \$15k: Garantia**

- Nenhuma → Alto

- Adequada → Moderado

- **\$15k a \$35k: Emprego**

- Desempregado → Alto

- Autônomo → Dívida

- Alta → Alto

- Baixa → Moderado

- Fixo → Idade

- Jovem → Moderado

- Adulto → História de crédito

- Boa → Baixo

- Ruim → Moderado

- **Acima de \$35k:** História de Crédito

Boa → Baixo

Desconhecida → Baixo

Ruim → Dívida

-Alta → Alto

-Baixa → Moderado

1.2.2 Algoritmo C4.5:

Para o algoritmo C4.5, substitui-se o critério do Ganho de Informação (Information Gain) pela Razão de Ganho (Gain Ratio). Esta métrica penaliza atributos com muitos valores distintos através do cálculo da Informação de Divisão (Split Information).

As fórmulas matemáticas base são:

$$SplitInfo(S, A) = - \sum_{i=1}^c \frac{|S_i|}{|S|} \log_2 \left(\frac{|S_i|}{|S|} \right)$$

$$GainRatio(S, A) = \frac{Gain(S, A)}{SplitInfo(S, A)}$$

1.2.2.1 Cálculos para o Nó Raiz

Utilizando os ganhos de informação (Gain) já calculados anteriormente para o nó inicial, calcula-se o SplitInfo e a GainRatio para cada atributo.

RENDA (\$0 a \$15k: 10, \$15k a \$35k: 9, Acima de \$35k: 11)

$$SplitInfo = - \left(\frac{10}{30} \log_2 \frac{10}{30} + \frac{9}{30} \log_2 \frac{9}{30} + \frac{11}{30} \log_2 \frac{11}{30} \right) \approx 1,5801$$

$$GainRatio = \frac{0,6551}{1,5801} = 0,4146$$

EMPREGO (Fixo: 18, Autônomo: 7, Desempregado: 5)

$$SplitInfo = - \left(\frac{18}{30} \log_2 \frac{18}{30} + \frac{7}{30} \log_2 \frac{7}{30} + \frac{5}{30} \log_2 \frac{5}{30} \right) \approx 1,3630$$

$$GainRatio = \frac{0,4294}{1,3630} \approx 0,3150$$

GARANTIA (Nenhuma: 20, Adequada: 10)

$$SplitInfo = - \left(\frac{20}{30} \log_2 \frac{20}{30} + \frac{10}{30} \log_2 \frac{10}{30} \right) \approx 0,9183$$

$$GainRatio = \frac{0,2881}{0,9183} = 0,3132$$

HISTÓRIA DE CRÉDITO (Boa: 11, Desconhecida: 10, Ruim: 9)

$$SplitInfo = - \left(\frac{11}{30} \log_2 \frac{11}{30} + \frac{10}{30} \log_2 \frac{10}{30} + \frac{9}{30} \log_2 \frac{9}{30} \right) \approx 1,5801$$

$$GainRatio = \frac{0,2552}{1,5801} = 0,1612$$

DÍVIDA (Alta: 15, Baixa: 15)

$$SplitInfo = - \left(\frac{15}{30} \log_2 \frac{15}{30} + \frac{15}{30} \log_2 \frac{15}{30} \right) = 1,0000$$

$$GainRatio = \frac{0,1966}{1,0000} = 0,1966$$

IDADE (Adulto: 18, Jovem: 12)

$$SplitInfo = - \left(\frac{18}{30} \log_2 \frac{18}{30} + \frac{12}{30} \log_2 \frac{12}{30} \right) \approx 0,9710$$

$$GainRatio = \frac{0,0685}{0,9710} = 0,0706$$

O ganho médio é 0,3152. Só Renda e Emprego o superam, e **Renda** tem a maior Razão de Ganho, então é a raiz.

Ramo 1: Renda = "\$0 a \$15k" (10 exemplos: 8 Alto, 2 Moderado)

Garantia separa perfeitamente os dados (Nenhuma → Alto; Adequada → Moderado). Seu ganho é 0,7219 e o SplitInfo também, logo a Razão de Ganho é 1,0000, a máxima possível. Esse é o mesmo resultado do ID3.

Ramo 2: Renda = "\$15k a \$35k" (9 exemplos: 2 Alto, 6 Moderado, 1 Baixo)

O ganho médio é 0,3330, e só Emprego (0,5172) e Dívida (0,3789) o superam.

- Emprego: SplitInfo = 1,3516, Razão de Ganho = 0,3827
- Dívida: SplitInfo = 0,9911, Razão de Ganho = 0,3823

Emprego vence por pequena margem, como no ID3.

- Desempregado (1 ex) → Alto
- Autônomo (E2, E17, E27): Dívida separa perfeitamente (razão de ganho 1,0): Alta → Alto; Baixa → Moderado

- Fixo (E3, E12, E15, E19, E22): Idade, História de Crédito e Garantia têm o mesmo ganho ($\approx 0,322$). Idade e Garantia dividem em 2 ramos (SplitInfo $\approx 0,971$) e têm Razão de Ganho $\approx 0,332$, acima da de História de Crédito ($\approx 0,212$). Idade e Garantia empatam, e adota-se Idade.
 - Jovem $\rightarrow$ Moderado (E3, E12, E22).
 - Adulto (E15 Baixo, E19 Moderado) ainda é impuro. História de Crédito o separa: Boa $\rightarrow$ Baixo (E15) e Ruim $\rightarrow$ Moderado (E19).

Ramo 3: Renda = "Acima de \$35k" (11 exemplos: 1 Alto, 1 Moderado, 9 Baixo)

Este subconjunto tem 11 exemplos (E5, E6, E8, E9, E10, E13, E18, E21, E23, E26, E28): 1 Alto, 1 Moderado e 9 Baixo, com $H(S) \approx 0,8659$.

Cálculo do Emprego: $H(\text{Fixo}) = 0,5033$, $H(\text{Autônomo}) = 0$ e $H(\text{Desempregado}) = 0$. A entropia ponderada é $(9/11 \times 0,5033) = 0,4118$, então o ganho é $0,8659 - 0,4118 = 0,4541$.

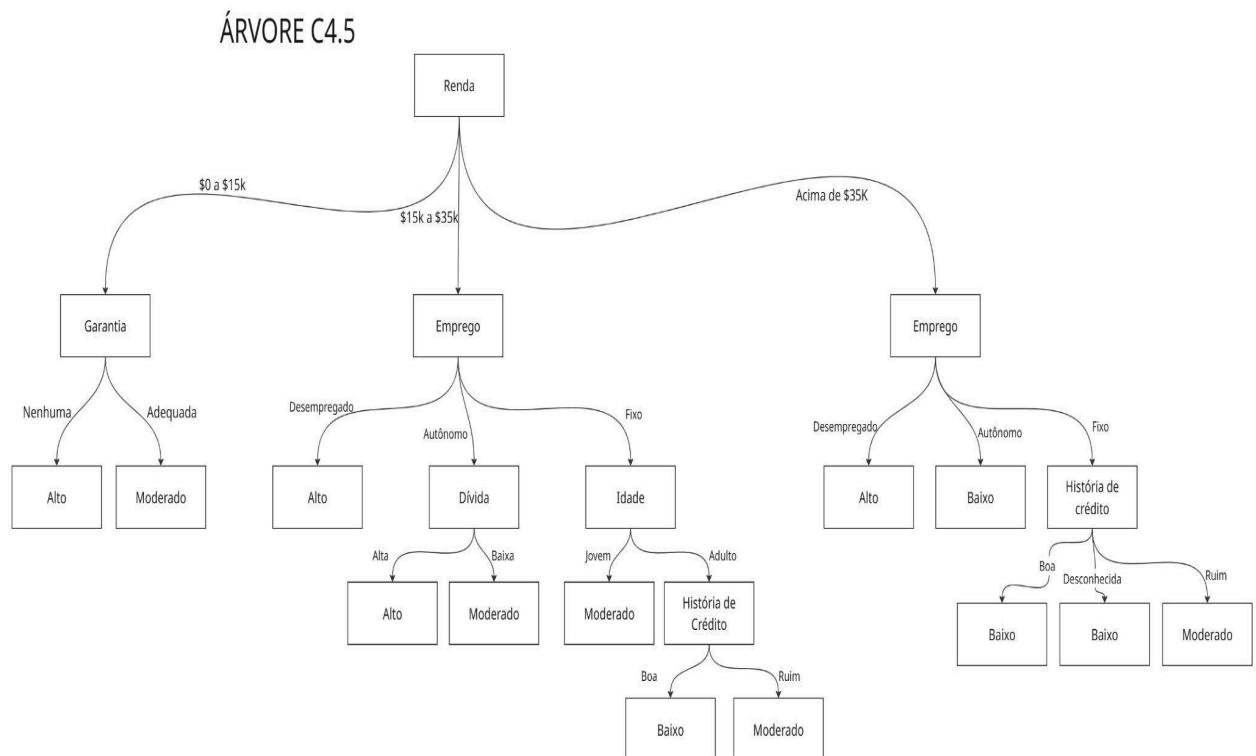
O ganho médio é $(0,6841 + 0,1831 + 0,1831 + 0,4541 + 0,0588) / 5 \approx 0,3126$. Apenas História de Crédito (0,6841) e Emprego (0,4541) o superam. Entre os dois, vence o de maior Razão de Ganho, que é Emprego (0,5245 contra 0,4576). O SplitInfo de História é maior porque ela se divide em subconjuntos mais equilibrados (5, 4 e 2), enquanto Emprego tem uma partição desbalanceada (9, 1 e 1).

Esse é o ponto em que o C4.5 diverge do ID3, o ID3 escolhe História de Crédito (maior ganho), e o C4.5 escolhe Emprego (maior razão).

- Desempregado (E28) $\rightarrow$ Alto (puro).
- Autônomo (E21) $\rightarrow$ Baixo (puro).
- Fixo (9 ex: 8 Baixo, 1 Moderado, $H = 0,5033$) $\rightarrow$ ainda impuro, precisa de nova ramificação.

Sub-ramo Emprego = Fixo (E5, E6, E8, E9, E10, E13, E18, E23, E26). O ganho médio é 0,1898 e só a História de Crédito o supera. Ela separa perfeitamente: Boa (E9, E10, E13, E18) $\rightarrow$ Baixo; Desconhecida (E5, E6, E23, E26) $\rightarrow$ Baixo; Ruim (E8) $\rightarrow$ Moderado.

A estrutura final da árvore C4.5 é a seguinte:



- Renda
 - \$0 a \$15k: Garantia
 - Nenhuma → Alto
 - Adequada → Moderado
 - \$15k a \$35k: Emprego
 - Desempregado → Alto
 - Autônomo → Dívida
 - Alta → Alto
 - Baixa → Moderado
 - Fixo → Idade
 - Jovem → Moderado
 - Adulto → História de Crédito
 - Boa → Baixo
 - Ruim → Moderado
 - Acima de \$35k: Emprego
 - Desempregado → Alto
 - Autônomo → Baixo
 - Fixo → História de Crédito
 - Boa → Baixo
 - Desconhecida → Baixo
 - Ruim → Moderado

1.2.3 Algoritmo CART:

O algoritmo CART (Classification and Regression Trees) constrói árvores estritamente binárias (decisões Sim/Não) baseadas na Impureza de Gini em vez do Ganho de Informação baseado na Entropia.

A fórmula da Impureza de Gini para um nó é calculada por:

$$Gini(S) = 1 - \sum p_i^2$$

A avaliação de um atributo A resulta na média ponderada da impureza dos dois subconjuntos gerados (Sim e Não):

$$Gini_A(S) = \frac{|S_{Sim}|}{|S|} Gini(S_{Sim}) + \frac{|S_{Não}|}{|S|} Gini(S_{Não})$$

O objetivo do algoritmo é encontrar a divisão binária que resulte no menor $Gini_A(S)$ possível.

1.2.3.1 Cálculos de Gini para o Nó Raiz

A partir dos dados da tabela com 30 exemplos (11 Alto, 9 Moderado, 10 Baixo), calcula-se o Gini inicial:

$$Gini(S_0) = 1 - \left[\left(\frac{11}{30} \right)^2 + \left(\frac{9}{30} \right)^2 + \left(\frac{10}{30} \right)^2 \right] = 1 - \left[\frac{121}{900} + \frac{81}{900} + \frac{100}{900} \right] \approx 0,6644$$

Como o CART apenas permite divisões binárias, os atributos com três categorias devem ser testados em combinações de um contra o resto. Abaixo estão os testes das principais divisões candidatas para a raiz:

- **Teste 1:** Renda == "Acima de \$35k"
 - **Sim (11 ex):** 1 Alto, 1 Moderado, 9 Baixo.

$$Gini_{sim} = 1 - \left[\left(\frac{1}{11} \right)^2 + \left(\frac{1}{11} \right)^2 + \left(\frac{9}{11} \right)^2 \right] = 1 - \frac{83}{121} \approx 0,3140$$
 - **Não (19 ex):** 10 Alto, 8 Moderado, 1 Baixo.

$$Gini_{\sim} = 1 - \left[\left(\frac{10}{19} \right)^2 + \left(\frac{8}{19} \right)^2 + \left(\frac{1}{19} \right)^2 \right] = 1 - \frac{165}{361} \approx 0,5429$$
 - **Gini Ponderado:** $\left(\frac{11}{30} \times 0,3140 \right) + \left(\frac{19}{30} \times 0,5429 \right) \approx 0,4589$
- **Teste 2:** Renda == "\$0 a \$15k"
 - **Sim (10 ex):** 8 Alto, 2 Moderado, 0 Baixo.

$$Gini_{sim} = 1 - \left[\left(\frac{8}{10} \right)^2 + \left(\frac{2}{10} \right)^2 \right] = 0,3200$$
 - **Não (20 ex):** 3 Alto, 7 Moderado, 10 Baixo.

$$Gini_{\sim} = 1 - \left[\left(\frac{3}{20} \right)^2 + \left(\frac{7}{20} \right)^2 + \left(\frac{10}{20} \right)^2 \right] = 0,6050$$
 - **Gini Ponderado:** $\left(\frac{10}{30} \times 0,3200 \right) + \left(\frac{20}{30} \times 0,6050 \right) \approx 0,5100$
- **Teste 3:** Emprego == "Fixo"

- **Sim (18 ex):** (18 ex: 2 Alto, 7 Moderado, 9 Baixo): $Gini \approx 0,5864$
 - **Não (12 ex):** (12 ex: 9 Alto, 2 Moderado, 1 Baixo): $Gini \approx 0,4028$
 - **Gini Ponderado:** $(18/30 \times 0,5864) + (12/30 \times 0,4028) \approx 0,5130$
- **Teste 4:** Garantia == "Adequada"
 - **Sim (10 ex):** 0 Alto, 5 Moderado, 5 Baixo.

$$Gini_{sim} = 1 - [(0,5)^2 + (0,5)^2] = 0,5000$$
 - **Não (20 ex):** 11 Alto, 4 Moderado, 5 Baixo.

$$Gini_{nao} \approx 1 - \left[\left(\frac{11}{20}\right)^2 + \left(\frac{4}{20}\right)^2 + \left(\frac{5}{20}\right)^2 \right] = 0,5950$$
 - **Gini Ponderado:** $\left(\frac{10}{30} \times 0,5000\right) + \left(\frac{20}{30} \times 0,5950\right) \approx 0,5633$

O menor Gini ponderado é alcançado com a divisão Renda == "Acima de \$35k" (0,4589). Este teste torna-se o nó raiz da árvore CART.

1.2.3.2 Nó 1: Renda = "Acima de \$35k" → SIM:

Contém 11 exemplos (E5, E6, E8, E9, E10, E13, E18, E21, E23, E26, E28): 9 Baixo, 1 Moderado e 1 Alto, com $Gini = 0,3140$.

- História de Crédito = "Ruim" = 0,0909
- Emprego = "Desempregado" = 0,1636
- Emprego = "Fixo" = 0,2525

História de Crédito = "Ruim" é a melhor divisão:

- **Não** (9 ex, todos Baixo; $Gini = 0$) → folha **BAIXO**.
- **Sim** (E8 Moderado e E28 Alto; $Gini = 0,5$). A divisão Dívida = "Alta" tem Gini ponderado 0 e produz folhas puras:
 - Dívida = "Alta" (E28) → **ALTO**
 - Dívida ≠ "Alta" (E8) → **MODERADO**

1.2.3.3: Nó 2: Renda = "Acima de \$35k" → NÃO

Contém 19 exemplos: 10 Alto, 8 Moderado e 1 Baixo, com $Gini = 0,5429$.

- Garantia = "Adequada" = 0,3850
- Dívida = "Alta" = 0,4023
- Emprego = "Fixo" = 0,4023
- Renda "\$0 a \$15k" = 0,4023

Garantia = "Adequada" é a melhor divisão: Sim (5 ex: 4 Moderado, 1 Baixo; $Gini = 0,3200$) e Não (14 ex: 10 Alto, 4 Moderado; $Gini = 0,4082$).

1.2.3.4: Nó 3: Garantia = "Adequada" → SIM

Contém 5 exemplos (E15, E17, E19, E25, E29): 4 Moderado e 1 Baixo, com $Gini = 0,3200$.

- História de crédito = "Boa" = 0,2
- História de crédito = "Desconhecida" = 0,2667
- Renda "\$0 a \$15k" = 0,2667

História de Crédito $\neq$ "Boa" (E17, E19, E29; todos Moderado) $\rightarrow$ folha MODERADO.

História de Crédito = "Boa" (E15 Baixo e E25 Moderado; Gini = 0,5). A divisão Renda = "\$15k a \$35k" separa perfeitamente:

- Renda = "\$15k a \$35k" (E15) $\rightarrow$ BAIXO
- Renda $\neq$ "\$15k a \$35k" (E25) $\rightarrow$ MODERADO

1.2.3.5: Nó 4: Garantia = "Adequada" $\rightarrow$ NÃO

Contém 14 exemplos: 10 Alto e 4 Moderado, com Gini = 0,4082.

- Renda = "\$15k a \$35k" = 0,1905
- Dívida = "Alta" = 0,2984
- Emprego = "Fixo" = 0,2984

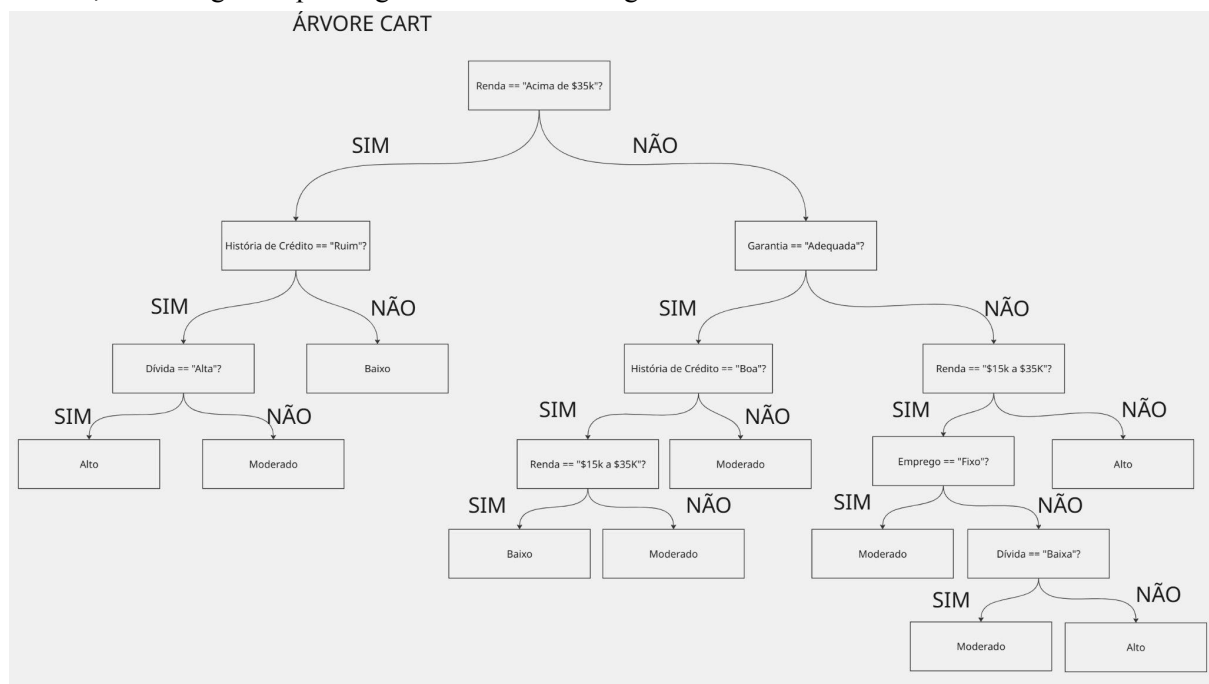
Renda = "\$15k a \$35k" é a melhor divisão (empatada com renda = "\$0 a \$15k", que gera a mesma partição):

Renda $\neq$ "\$15k a \$35k" (8 ex, todos Alto; Gini = 0) $\rightarrow$ folha ALTO.

Renda = "\$15k a \$35k" (E2, E3, E12, E14, E22, E27: 2 Alto e 4 Moderado; Gini = 0,4444). Aqui Emprego = "Fixo" e Dívida = "Alta" empatam com Gini ponderado 0,2222, e foi adotado Emprego = "Fixo":

- Emprego = "Fixo" (E3, E12, E22; todos Moderado) $\rightarrow$ folha MODERADO.
- Emprego $\neq$ "Fixo" (E2 Alto, E14 Alto, E27 Moderado; Gini = 0,4444). A divisão Dívida = "Baixa" separa perfeitamente:
 - Dívida = "Baixa" (E27) $\rightarrow$ MODERADO
 - Dívida $\neq$ "Baixa" (E2, E14) $\rightarrow$ ALTO

Por fim, a árvore gerada pelo algoritmo CART é a seguinte:



- Renda == "Acima de \$35k"?
 - SIM $\rightarrow$ História de Crédito == "Ruim"?
 - SIM $\rightarrow$ Dívida == "Alta"?
 - SIM $\rightarrow$ Classe ALTO

-NÃO → Classe MODERADO
 -NÃO → Classe BAIXO
 NÃO → Garantia == "Adequada"?
 -SIM → História de Crédito == "Boa"?
 -SIM → Renda == "\$15k a \$35k"?
 -SIM → Classe BAIXO
 -NÃO → Classe MODERADO
 -NÃO → Classe MODERADO
 -NÃO → Renda == "\$15k a \$35k"?
 -SIM → Emprego == "Fixo"?
 -SIM → Classe MODERADO
 -NÃO → Dívida == "Baixa"?
 -SIM → Classe MODERADO
 -NÃO → Classe ALTO
 -NÃO → Classe ALTO

1.3 (iii) Regras do tipo SE...ENTÃO...

1.3.1 Base de Conhecimento da Árvore ID3

- **R1:** SE Renda = "\$0 a \$15k" E Garantia = "Nenhuma" ENTÃO Risco = Alto
- **R2:** SE Renda = "\$0 a \$15k" E Garantia = "Adequada" ENTÃO Risco = Moderado
- **R3:** SE Renda = "\$15k a \$35k" E Emprego = "Desempregado" ENTÃO Risco = Alto
- **R4:** SE Renda = "\$15k a \$35k" E Emprego = "Autônomo" E Dívida = "Alta" ENTÃO Risco = Alto
- **R5:** SE Renda = "\$15k a \$35k" E Emprego = "Autônomo" E Dívida = "Baixa" ENTÃO Risco = Moderado
- **R6:** SE Renda = "\$15k a \$35k" E Emprego = "Fixo" E Idade = "Jovem" ENTÃO Risco = Moderado
- **R7:** SE Renda = "\$15k a \$35k" E Emprego = "Fixo" E Idade = "Adulto" E História de Crédito = "Boa" ENTÃO Risco = Baixo
- **R8:** SE Renda = "\$15k a \$35k" E Emprego = "Fixo" E Idade = "Adulto" E História de Crédito = "Ruim" ENTÃO Risco = Moderado
- **R9:** SE Renda = "Acima de \$35k" E História de Crédito = "Boa" ENTÃO Risco = Baixo
- **R10:** SE Renda = "Acima de \$35k" E História de Crédito = "Desconhecida" ENTÃO Risco = Baixo
- **R11:** SE Renda = "Acima de \$35k" E História de Crédito = "Ruim" E Dívida = "Alta" ENTÃO Risco = Alto
- **R12:** SE Renda = "Acima de \$35k" E História de Crédito = "Ruim" E Dívida = "Baixa" ENTÃO Risco = Moderado

1.3.2. Base de Conhecimento da Árvore C4.5

- **R1:** SE Renda = "\$0 a \$15k" E Garantia = "Nenhuma" ENTÃO Risco = Alto
- **R2:** SE Renda = "\$0 a \$15k" E Garantia = "Adequada" ENTÃO Risco = Moderado
- **R3:** SE Renda = "\$15k a \$35k" E Emprego = "Desempregado" ENTÃO Risco = Alto

- **R4:** SE Renda = "\$15k a \$35k" E Emprego = "Autônomo" E Dívida = "Alta" ENTÃO Risco = Alto
- **R5:** SE Renda = "\$15k a \$35k" E Emprego = "Autônomo" E Dívida = "Baixa" ENTÃO Risco = Moderado
- **R6:** SE Renda = "\$15k a \$35k" E Emprego = "Fixo" E Idade = "Jovem" ENTÃO Risco = Moderado
- **R7:** SE Renda = "\$15k a \$35k" E Emprego = "Fixo" E Idade = "Adulto" E História de Crédito = "Boa" ENTÃO Risco = Baixo
- **R8:** SE Renda = "\$15k a \$35k" E Emprego = "Fixo" E Idade = "Adulto" E História de Crédito = "Ruim" ENTÃO Risco = Moderado
- **R9:** SE Renda = "Acima de \$35k" E Emprego = "Desempregado" ENTÃO Risco = Alto
- **R10:** SE Renda = "Acima de \$35k" E Emprego = "Autônomo" ENTÃO Risco = Baixo
- **R11:** SE Renda = "Acima de \$35k" E Emprego = "Fixo" E História de Crédito = "Boa" ENTÃO Risco = Baixo
- **R12:** SE Renda = "Acima de \$35k" E Emprego = "Fixo" E História de Crédito = "Desconhecida" ENTÃO Risco = Baixo
- **R13:** SE Renda = "Acima de \$35k" E Emprego = "Fixo" E História de Crédito = "Ruim" ENTÃO Risco = Moderado

1.3.3 Base de Conhecimento da Árvore CART

- **R1:** SE Renda = "Acima de \$35k" E História de Crédito = "Ruim" E Dívida = "Alta" ENTÃO Risco = Alto
- **R2:** SE Renda = "Acima de \$35k" E História de Crédito = "Ruim" E Dívida != "Alta" ENTÃO Risco = Moderado
- **R3:** SE Renda = "Acima de \$35k" E História de Crédito != "Ruim" ENTÃO Risco = Baixo
- **R4:** SE Renda != "Acima de \$35k" E Garantia = "Adequada" E História de Crédito = "Boa" E Renda = "\$15k a \$35k" ENTÃO Risco = Baixo
- **R5:** SE Renda != "Acima de \$35k" E Garantia="Adequada" E História de Crédito="Boa" E Renda!="\$15k a \$35k" ENTÃO Risco = Moderado
- **R6:** SE Renda != "Acima de \$35k" E Garantia = "Adequada" E História de Crédito != "Boa" ENTÃO Risco = Moderado
- **R7:** SE Renda != "Acima de \$35k" E Garantia!="Adequada" E Renda="\$15k a \$35k" E Emprego = "Fixo" ENTÃO Risco = Moderado
- **R8:** SE Renda != "Acima de \$35k" E Garantia!="Adequada" E Renda="\$15k a \$35k" E Emprego != "Fixo" E Dívida = "Baixa" ENTÃO Risco = Moderado
- **R9:** SE Renda != "Acima de \$35k" E Garantia!="Adequada" E Renda="\$15k a \$35k" E Emprego != "Fixo" E Dívida != "Baixa" ENTÃO Risco = Alto
- **R10:** SE Renda != "Acima de \$35k" E Garantia!="Adequada" E Renda!="\$15k a \$35k" ENTÃO Risco = Alto

1.4 (iv) Comparação das bases:

1.4.1 ID3 e C4.5:

Ambas usam partições de múltiplas ramificações e condições afirmativas, que mapeiam diretamente a natureza categórica dos atributos (ex.: Renda = "\$15k a \$35k").

As regras R1 a R8 são idênticas. A diferença está no ramo de renda acima de \$35k: o ID3 usa História de Crédito e Dívida (R9 a R12), e o C4.5 usa Emprego e, para emprego fixo, História de Crédito (R9 a R13). O ID3 é um pouco mais enxuto (12 contra 13 regras).

1.4.2 CART:

Tem menos regras, mas condições mais longas, com negações e exclusões sucessivas (por exemplo, Renda $\neq$ "Acima de \$35k" E Garantia $\neq$ "Adequada"). Isso dificulta a leitura por um analista e a geração de explicações ao cliente.

1.4.3 Escolha:

A escolha é a base do C4.5, pelos seguintes motivos:

- **Explicabilidade:** Em sistemas financeiros, o sistema precisa justificar a decisão ao cliente. As regras do C4.5 permitem uma explicação direta, por exemplo: "Seu crédito foi considerado de alto risco porque sua renda está entre \$15k e \$35k e você está desempregado".
- **Critério de seleção mais robusto:** A Razão de Ganho corrige o viés do ganho de informação a favor de atributos com muitos valores, o que gera árvores menos enviesadas.
- **Regras sem dupla negação:** O CART depende de negações e exclusões que dificultam a tradução para linguagem natural e prejudicam o módulo de explicação.
- **Consistência do conhecimento:** Nos dados, os 5 exemplos com Emprego = "Desempregado" têm Risco = Alto. A regra R9 do C4.5 (renda acima de \$35k e desempregado $\rightarrow$ Alto) reaparece nesse padrão, em linha com R3 (renda de \$15k a \$35k e desempregado $\rightarrow$ Alto). Embora R9 se apoie em um único exemplo (E28) no seu ramo, ela é coerente com um padrão observado nos demais ramos. No ID3, o mesmo exemplo cai em R11 (história ruim e dívida alta), uma regra que só se justifica por esse exemplo.

2. Questão 2(<https://github.com/luctrindade/LISTA-1-DE-IA-AB1.git>):

2.1 Ferramentas e Configuração

Os algoritmos foram aplicados à base ampliada de 30 exemplos da Questão 1 (composta por 6 atributos preditivos e 3 classes), sem aplicação de poda e sem limite de profundidade.

- **ID3 e C4.5:** Foi utilizada a biblioteca `chefboost` (Python), que implementa as árvores com divisões de um ramo por valor sobre atributos categóricos. O critério adotado pelo ID3 é o Ganho de Informação, enquanto o C4.5 utiliza a Razão de Ganho. A ferramenta permite a configuração de ambos com uma única chamada (`{"algorithm": "ID3"}` e `{"algorithm": "C4.5"}`). Os caminhos `else`, gerados pela ferramenta como padrão de segurança (mas que não correspondem a exemplos reais da base), foram descartados durante a extração das regras.
- **CART:** Foi utilizada a biblioteca `scikit-learn`, instanciando o modelo `DecisionTreeClassifier(criterion="gini", random_state=42)`. Como o algoritmo exige entradas numéricas, aplicou-se o one-hot encoding aos atributos categóricos. Devido a isso, cada teste na árvore é do tipo "atributo = valor? Sim/Não", o que caracteriza uma divisão estritamente binária, propriedade definidora do CART.

2.2 Resultados: ID3 (chefboost)

- **Acurácia no treinamento:** 100% (matriz de confusão diagonal: 11 Alto, 9 Moderado, 10 Baixo).
- **Total de regras geradas:** 13 regras.

Estrutura da Árvore:

- Renda
 - \$0 a \$15k: Garantia
 - Nenhuma → Alto
 - Adequada → Moderado
 - \$15k a \$35k: Emprego
 - Desempregado → Alto
 - Autônomo → Dívida
 - Alta → Alto
 - Baixa → Moderado
 - Fixo → História de Crédito
 - Boa → Dívida
 - Alta → Moderado
 - Baixa → Baixo
 - Desconhecida → Moderado
 - Ruim → Moderado
 - Acima de \$35k: História de Crédito
 - Boa → Baixo
 - Desconhecida → Baixo
 - Ruim → Dívida
 - Alta → Alto
 - Baixa → Moderado

Base de Regras Extraída:

- **R1:** SE Renda = "Acima de \$35k" E História de Crédito = "Boa" ENTÃO Risco = Baixo
- **R2:** SE Renda = "Acima de \$35k" E História de Crédito = "Desconhecida" ENTÃO Risco = Baixo
- **R3:** SE Renda = "Acima de \$35k" E História de Crédito = "Ruim" E Dívida = "Baixa" ENTÃO Risco = Moderado
- **R4:** SE Renda = "Acima de \$35k" E História de Crédito = "Ruim" E Dívida = "Alta" ENTÃO Risco = Alto
- **R5:** SE Renda = "\$0 a \$15k" E Garantia = "Nenhuma" ENTÃO Risco = Alto
- **R6:** SE Renda = "\$0 a \$15k" E Garantia = "Adequada" ENTÃO Risco = Moderado
- **R7:** SE Renda = "\$15k a \$35k" E Emprego = "Fixo" E História de Crédito = "Boa" E Dívida = "Alta" ENTÃO Risco = Moderado
- **R8:** SE Renda = "\$15k a \$35k" E Emprego = "Fixo" E História de Crédito = "Boa" E Dívida = "Baixa" ENTÃO Risco = Baixo
- **R9:** SE Renda = "\$15k a \$35k" E Emprego = "Fixo" E História de Crédito = "Ruim" ENTÃO Risco = Moderado

- **R10:** SE Renda = "\$15k a \$35k" E Emprego = "Fixo" E História de Crédito = "Desconhecida" ENTÃO Risco = Moderado
- **R11:** SE Renda = "\$15k a \$35k" E Emprego = "Autônomo" E Dívida = "Baixa" ENTÃO Risco = Moderado
- **R12:** SE Renda = "\$15k a \$35k" E Emprego = "Autônomo" E Dívida = "Alta" ENTÃO Risco = Alto
- **R13:** SE Renda = "\$15k a \$35k" E Emprego = "Desempregado" ENTÃO Risco = Alto

2.3 Resultados: C4.5 (chefboost)

- **Acurácia no treinamento:** 100% (matriz de confusão diagonal).
- **Total de regras geradas:** 13 regras.

Estrutura da Árvore:

- Renda
 - \$0 a \$15k: Garantia
 - Nenhuma → Alto
 - Adequada → Moderado
 - \$15k a \$35k: Emprego
 - Desempregado → Alto
 - Autônomo → Dívida
 - Alta → Alto
 - Baixa → Moderado
 - Fixo → Garantia
 - Nenhuma → Moderado
 - Adequada → História de Crédito
 - Boa → Baixo
 - Ruim → Moderado
 - Acima de \$35k: Emprego
 - Desempregado → Alto
 - Autônomo → Baixo
 - Fixo → História de Crédito
 - Boa → Baixo
 - Desconhecida → Baixo
 - Ruim → Moderado

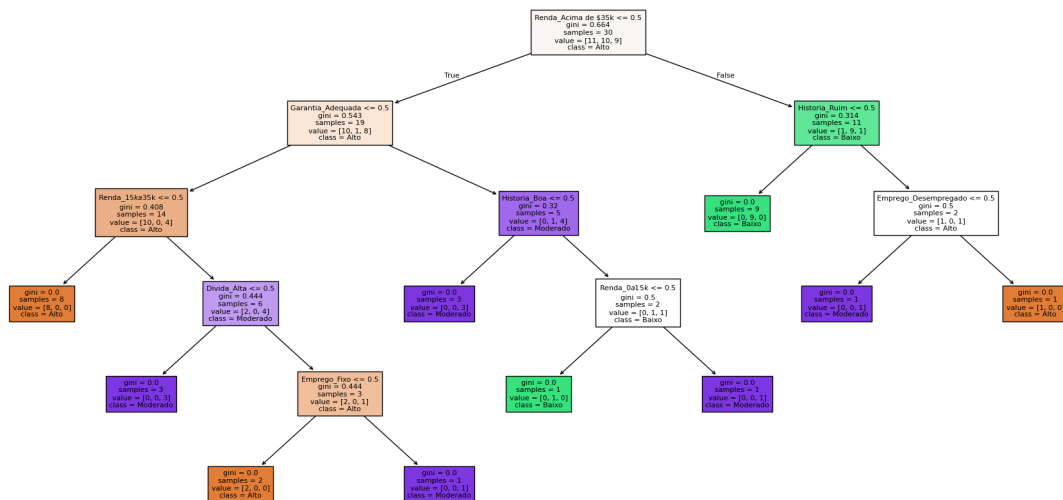
Base de Regras Extraída:

- **R1:** SE Renda = "Acima de \$35k" E Emprego = "Fixo" E História de Crédito = "Desconhecida" ENTÃO Risco = Baixo
- **R2:** SE Renda = "Acima de \$35k" E Emprego = "Fixo" E História de Crédito = "Boa" ENTÃO Risco = Baixo
- **R3:** SE Renda = "Acima de \$35k" E Emprego = "Fixo" E História de Crédito = "Ruim" ENTÃO Risco = Moderado
- **R4:** SE Renda = "Acima de \$35k" E Emprego = "Autônomo" ENTÃO Risco = Baixo
- **R5:** SE Renda = "Acima de \$35k" E Emprego = "Desempregado" ENTÃO Risco = Alto
- **R6:** SE Renda = "\$0 a \$15k" E Garantia = "Nenhuma" ENTÃO Risco = Alto

- **R7:** SE Renda = "\$0 a \$15k" E Garantia = "Adequada" ENTÃO Risco = Moderado
- **R8:** SE Renda = "\$15k a \$35k" E Emprego = "Fixo" E Garantia = "Nenhuma" ENTÃO Risco = Moderado
- **R9:** SE Renda = "\$15k a \$35k" E Emprego = "Fixo" E Garantia = "Adequada" E História de Crédito = "Boa" ENTÃO Risco = Baixo
- **R10:** SE Renda = "\$15k a \$35k" E Emprego = "Fixo" E Garantia = "Adequada" E História de Crédito = "Ruim" ENTÃO Risco = Moderado
- **R11:** SE Renda = "\$15k a \$35k" E Emprego = "Autônomo" E Dívida = "Baixa" ENTÃO Risco = Moderado
- **R12:** SE Renda = "\$15k a \$35k" E Emprego = "Autônomo" E Dívida = "Alta" ENTÃO Risco = Alto
- **R13:** SE Renda = "\$15k a \$35k" E Emprego = "Desempregado" ENTÃO Risco = Alto

2.4 Resultados: CART (scikit-learn)

- **Acurácia no treinamento:** 100%.
- **Métricas estruturais:** 10 folhas, profundidade 5.



Estrutura da Árvore:

- Renda = "Acima de \$35k"?
 - SIM → História de Crédito = "Ruim"?
 - SIM → Emprego = "Desempregado"?
 - SIM → Alto
 - NÃO → Moderado
 - NÃO → Baixo
 - NÃO → Garantia = "Adequada"?
 - SIM → História de Crédito = "Boa"?
 - SIM → Renda = "\$0 a \$15k"?

- SIM → Moderado
 - NÃO → Baixo
- NÃO → Moderado
- NÃO → Renda = "\$15k a \$35k"?
 - NÃO → Alto
 - SIM → Dívida = "Alta"?
 - NÃO → Moderado
 - SIM → Emprego = "Fixo"?
 - SIM → Moderado
 - NÃO → Alto

Base de Regras Extraída:

- **R1:** SE Renda ≠ "Acima de \$35k" E Garantia ≠ "Adequada" E Renda ≠ "\$15k a \$35k" ENTÃO Risco = Alto
- **R2:** SE Renda = "\$15k a \$35k" E Garantia ≠ "Adequada" E Dívida ≠ "Alta" ENTÃO Risco = Moderado
- **R3:** SE Renda = "\$15k a \$35k" E Garantia ≠ "Adequada" E Dívida = "Alta" E Emprego ≠ "Fixo" ENTÃO Risco = Alto
- **R4:** SE Renda = "\$15k a \$35k" E Garantia ≠ "Adequada" E Dívida = "Alta" E Emprego = "Fixo" ENTÃO Risco = Moderado
- **R5:** SE Renda ≠ "Acima de \$35k" E Garantia = "Adequada" E História de Crédito ≠ "Boa" ENTÃO Risco = Moderado
- **R6:** SE Renda ≠ "Acima de \$35k" E Garantia = "Adequada" E História de Crédito = "Boa" E Renda ≠ "\$0 a \$15k" ENTÃO Risco = Baixo
- **R7:** SE Renda = "\$0 a \$15k" E Garantia = "Adequada" E História de Crédito = "Boa" ENTÃO Risco = Moderado
- **R8:** SE Renda = "Acima de \$35k" E História de Crédito ≠ "Ruim" ENTÃO Risco = Baixo
- **R9:** SE Renda = "Acima de \$35k" E História de Crédito = "Ruim" E Emprego ≠ "Desempregado" ENTÃO Risco = Moderado
- **R10:** SE Renda = "Acima de \$35k" E História de Crédito = "Ruim" E Emprego = "Desempregado" ENTÃO Risco = Alto

Todas as árvores geradas pelas ferramentas atingiram 100% de acurácia no treino, assim como as árvores manuais. As diferenças estruturais vêm de critérios de desempate:

ID3: no ramo Renda = "\$15k a \$35k" e Emprego = "Fixo" (5 exemplos), Idade, História de Crédito e Garantia empatam com o mesmo ganho de informação ($\approx 0,322$). Na construção manual foi adotada a Idade; o chefboost escolheu a História de Crédito (seguida de Dívida). O número de regras muda de 12 para 13, mas a classificação dos 30 exemplos é a mesma.

C4.5: no mesmo ramo, Idade e Garantia empatam na Razão de Ganho ($\approx 0,332$). Manualmente foi adotada a Idade; o chefboost escolheu a Garantia.

2.5 Saída no terminal:

```
===== ID3 (chefboost) =====
26-10-08 16:10:37 - ID3 tree is going to be built...
26-10-08 16:10:41 - -----
26-10-08 16:10:41 - finished in 4.423981189727783 seconds
26-10-08 16:10:41 - -----
26-10-08 16:10:41 - Evaluate train set
26-10-08 16:10:41 - -----
26-10-08 16:10:41 - Accuracy: 100.0% on 30 instances
26-10-08 16:10:41 - Labels: ['Alto' 'Moderado' 'Baixo']
26-10-08 16:10:41 - Confusion matrix: [[11, 0, 0], [0, 9, 0], [0, 0, 10]]
26-10-08 16:10:41 - Decision Alto
26-10-08 16:10:41 - Accuracy: 100.0
26-10-08 16:10:41 - Precision: 100.0%, Recall: 100.0%, F1: 100.0%
26-10-08 16:10:41 - Decision Moderado
26-10-08 16:10:41 - Accuracy: 100.0
26-10-08 16:10:41 - Precision: 100.0%, Recall: 100.0%, F1: 100.0%
26-10-08 16:10:41 - Decision Baixo
26-10-08 16:10:41 - Accuracy: 100.0
26-10-08 16:10:41 - Precision: 100.0%, Recall: 100.0%, F1: 100.0%

--- Regras ID3 ---
R1: SE Renda = "Acima de $35k" E História de Crédito = "Boa" ENTÃO Risco = Baixo
R2: SE Renda = "Acima de $35k" E História de Crédito = "Desconhecida" ENTÃO Risco = Baixo
R3: SE Renda = "Acima de $35k" E História de Crédito = "Ruim" E Dívida = "Baixa" ENTÃO Risco = Moderado
R4: SE Renda = "Acima de $35k" E História de Crédito = "Ruim" E Dívida = "Alta" ENTÃO Risco = Alto
R5: SE Renda = "$0 a $15k" E Garantia = "Nenhuma" ENTÃO Risco = Alto
R6: SE Renda = "$0 a $15k" E Garantia = "Adequada" ENTÃO Risco = Moderado
R7: SE Renda = "$15k a $35k" E Emprego = "Fixo" E História de Crédito = "Boa" E Dívida = "Alta" ENTÃO Risco = Moderado
R8: SE Renda = "$15k a $35k" E Emprego = "Fixo" E História de Crédito = "Boa" E Dívida = "Baixa" ENTÃO Risco = Baixo
R9: SE Renda = "$15k a $35k" E Emprego = "Fixo" E História de Crédito = "Ruim" ENTÃO Risco = Moderado
R10: SE Renda = "$15k a $35k" E Emprego = "Fixo" E História de Crédito = "Desconhecida" ENTÃO Risco = Moderado
R11: SE Renda = "$15k a $35k" E Emprego = "Autônomo" E Dívida = "Baixa" ENTÃO Risco = Moderado
R12: SE Renda = "$15k a $35k" E Emprego = "Autônomo" E Dívida = "Alta" ENTÃO Risco = Alto
R13: SE Renda = "$15k a $35k" E Emprego = "Desempregado" ENTÃO Risco = Alto
Regras: 13 | Acurácia no treino: 100.00%

===== C4.5 (chefboost) =====
26-10-08 16:10:41 - C4.5 tree is going to be built...
26-10-08 16:10:46 - -----
26-10-08 16:10:46 - finished in 5.319246530532837 seconds
26-10-08 16:10:46 - -----
26-10-08 16:10:46 - Evaluate train set
26-10-08 16:10:46 - -----
26-10-08 16:10:46 - Accuracy: 100.0% on 30 instances
26-10-08 16:10:46 - Labels: ['Alto' 'Moderado' 'Baixo']
26-10-08 16:10:46 - Confusion matrix: [[11, 0, 0], [0, 9, 0], [0, 0, 10]]
26-10-08 16:10:46 - Decision Alto
26-10-08 16:10:46 - Accuracy: 100.0
26-10-08 16:10:46 - Precision: 100.0%, Recall: 100.0%, F1: 100.0%
26-10-08 16:10:46 - Decision Moderado
26-10-08 16:10:46 - Accuracy: 100.0
26-10-08 16:10:46 - Precision: 100.0%, Recall: 100.0%, F1: 100.0%
26-10-08 16:10:46 - Decision Baixo
26-10-08 16:10:46 - Accuracy: 100.0
26-10-08 16:10:46 - Precision: 100.0%, Recall: 100.0%, F1: 100.0%
```

```

--- Regras C4.5 ---
R1: SE Renda = "Acima de $35k" E Emprego = "Fixo" E História de Crédito = "Desconhecida" ENTÃO Risco = Baixo
R2: SE Renda = "Acima de $35k" E Emprego = "Fixo" E História de Crédito = "Boa" ENTÃO Risco = Baixo
R3: SE Renda = "Acima de $35k" E Emprego = "Fixo" E História de Crédito = "Ruim" ENTÃO Risco = Moderado
R4: SE Renda = "Acima de $35k" E Emprego = "Autônomo" ENTÃO Risco = Baixo
R5: SE Renda = "Acima de $35k" E Emprego = "Desempregado" ENTÃO Risco = Alto
R6: SE Renda = "$0 a $15k" E Garantia = "Nenhuma" ENTÃO Risco = Alto
R7: SE Renda = "$0 a $15k" E Garantia = "Adequada" ENTÃO Risco = Moderado
R8: SE Renda = "$15k a $35k" E Emprego = "Fixo" E Garantia = "Nenhuma" ENTÃO Risco = Moderado
R9: SE Renda = "$15k a $35k" E Emprego = "Fixo" E Garantia = "Adequada" E História de Crédito = "Boa" ENTÃO Risco = Baixo
R10: SE Renda = "$15k a $35k" E Emprego = "Fixo" E Garantia = "Adequada" E História de Crédito = "Ruim" ENTÃO Risco = Moderado
R11: SE Renda = "$15k a $35k" E Emprego = "Autônomo" E Dívida = "Baixa" ENTÃO Risco = Moderado
R12: SE Renda = "$15k a $35k" E Emprego = "Autônomo" E Dívida = "Alta" ENTÃO Risco = Alto
R13: SE Renda = "$15k a $35k" E Emprego = "Desempregado" ENTÃO Risco = Alto
Regras: 13 | Acurácia no treino: 100.00%

```

```

===== CART (scikit-learn) =====
R1: SE Renda ≠ "Acima de $35k" E Garantia ≠ "Adequada" E Renda ≠ "$15k a $35k" ENTÃO Risco = Alto
R2: SE Renda = "Acima de $35k" E Garantia ≠ "Adequada" E Renda = "$15k a $35k" E Dívida ≠ "Alta" ENTÃO Risco = Moderado
R3: SE Renda ≠ "Acima de $35k" E Garantia ≠ "Adequada" E Renda = "$15k a $35k" E Dívida = "Alta" E Emprego ≠ "Fixo" ENTÃO Risco = Alto
R4: SE Renda ≠ "Acima de $35k" E Garantia ≠ "Adequada" E Renda = "$15k a $35k" E Dívida = "Alta" E Emprego = "Fixo" ENTÃO Risco = Moderado
R5: SE Renda ≠ "Acima de $35k" E Garantia = "Adequada" E História de Crédito ≠ "Boa" ENTÃO Risco = Moderado
R6: SE Renda ≠ "Acima de $35k" E Garantia = "Adequada" E História de Crédito = "Boa" E Renda ≠ "$0 a $15k" ENTÃO Risco = Baixo
R7: SE Renda ≠ "Acima de $35k" E Garantia = "Adequada" E História de Crédito = "Boa" E Renda = "$0 a $15k" ENTÃO Risco = Moderado
R8: SE Renda = "Acima de $35k" E História de Crédito ≠ "Ruim" ENTÃO Risco = Baixo
R9: SE Renda = "Acima de $35k" E História de Crédito = "Ruim" E Emprego ≠ "Desempregado" ENTÃO Risco = Moderado
R10: SE Renda = "Acima de $35k" E História de Crédito = "Ruim" E Emprego = "Desempregado" ENTÃO Risco = Alto
Regras: 10 | Folhas: 10 | Profundidade: 5 | Acurácia no treino: 100.00%

```

```

RESUMO FINAL
=====
ID3 : 13 regras | Acurácia no treino: 100.00%
C4.5 : 13 regras | Acurácia no treino: 100.00%
CART : 10 regras | Acurácia no treino: 100.00%

```

2.6 APÊNDICE:

```

import pandas as pd

import re, shutil

import matplotlib.pyplot as plt

from chefboost import Chefboost as chef

from sklearn.tree import DecisionTreeClassifier, plot_tree

FEATURES = ["Historia", "Dívida", "Garantia", "Renda", "Emprego",
"Idade"]

NOMES = {"Historia": "História de Crédito", "Dívida": "Dívida"}

df = pd.read_csv("arquivo.csv")[FEATURES + ["Risco"]]

df_cb = df.rename(columns={"Risco": "Decision"})

def nome(f):

```

```

return NOMES.get(f, f)

# conversão de regras geradas em se...então

def regras_chefboost(caminho):

    regras, pilha = [], []

    for linha in open(caminho, encoding="utf-8"):

        codigo = linha.split("#")[0].rstrip()

        t = codigo.strip()

        if not t:

            continue

        ind = len(codigo) - len(t)

        if t.startswith(("if ", "elif ", "else")):

            while pilha and pilha[-1][0] >= ind:

                pilha.pop()

            m = re.findall(r"obj\[ (\d+) \] \s*==\s*['\"] ([^\"]*) [^\"]",
t)

            cond = " E ".join(f'{nome(FEATURES[int(i)])} = "{v}"' for
i, v in m) if m else None

            pilha.append((ind, cond))

        elif t.startswith("return"):

            while pilha and pilha[-1][0] >= ind:

                pilha.pop()

            classe = re.findall(r"return\s+['\"] ([^\"]*) [^\"]", t)[0]

            conds = [c for _, c in pilha]

```

```

        if None in conds:

            continue

        regras.append((conds, classe))

    return regras

# id3 e c4.5

resumo = {}

for alg in ["ID3", "C4.5"]:

    print(f"\n{'=' * 15} {alg} (chefboost) {'=' * 15}")

    config = {"algorithm": alg}

    try:

        modelo = chef.fit(df_cb.copy(), config=config,
target_label="Decision")

    except TypeError:

        modelo = chef.fit(df_cb.copy(), config=config)

    arq = f"rules_{alg.replace('.', ' ').py"

    shutil.copy("outputs/rules/rules.py", arq)

    print(f"\n--- Regras {alg} ---")

    rs = regras_chefboost(arq)

```

```

        for i, (conds, classe) in enumerate(rs, 1):

            print(f"R{i}: SE " + " E ".join(conds) + f" ENTÃO Risco = {classe}")

        acertos = sum(chef.predict(modelo, list(r[FEATURES])) == r["Risco"]
for _, r in df.iterrows())

        resumo[alg] = (len(rs), acertos / len(df))

        print(f"Regras: {len(rs)} | Acurácia no treino: {acertos / len(df) :.2%}")

# CART

X = pd.get_dummies(df[FEATURES]).astype(int)

y = df["Risco"]

cart = DecisionTreeClassifier(criterion="gini", random_state=42).fit(X, y)

nomes_cols = list(X.columns)

def regras_sklearn(clf, nomes_cols):

    t = clf.tree_

    def rec(no, cond):

        if t.children_left[no] == -1:

            yield cond, clf.classes_[t.value[no][0].argmax()]

            return

        attr_bruto, val = nomes_cols[t.feature[no]].split("_", 1)

        attr_formatado = nome(attr_bruto)

```

```

        yield from rec(t.children_left[no], cond + [f'{attr_formatado}
≠ "{val}"'])

        yield from rec(t.children_right[no], cond + [f'{attr_formatado}
= "{val}"'])

    return list(rec(0, []))

print(f"\n{'=' * 15} CART (scikit-learn) {'=' * 15}")

rs_cart = regras_sklearn(cart, nomes_cols)

for i, (conds, classe) in enumerate(rs_cart, 1):

    print(f"R{i}: SE " + " E ".join(conds) + f" ENTÃO Risco =
{classe}")

acuracia_cart = cart.score(X, y)

print(f"Regras: {len(rs_cart)} | Folhas: {cart.get_n_leaves()} |
Profundidade: {cart.get_depth()} | ")

    f"Acurácia no treino: {acuracia_cart:.2%}")

resumo["CART"] = (len(rs_cart), acuracia_cart)

# desenho da árvore CART

plt.figure(figsize=(24, 11))

plot_tree(cart, feature_names=nomes_cols,
class_names=list(cart.classes_), filled=True, fontsize=8)

plt.savefig("cart.png", dpi=150, bbox_inches="tight")

```

```
print("\n")

print(" "*13 + "RESUMO FINAL")

print("="*40)

for alg, (n, acc) in resumo.items():

    print(f"{alg:<5} : {n} regras | Acurácia no treino: {acc:.2%}")
```

3. Questão 3(<https://github.com/luctrindade/LISTA-1-DE-IA-AB1.git>):

Para a resolução desta questão, foi selecionada a base de dados Gestão de Recursos Humanos (IBM HR Analytics Employee Attrition & Performance), que é amplamente utilizada no Kaggle para prever a rotatividade de funcionários (employee turnover ou attrition). (disponível no Kaggle em: <https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset>)

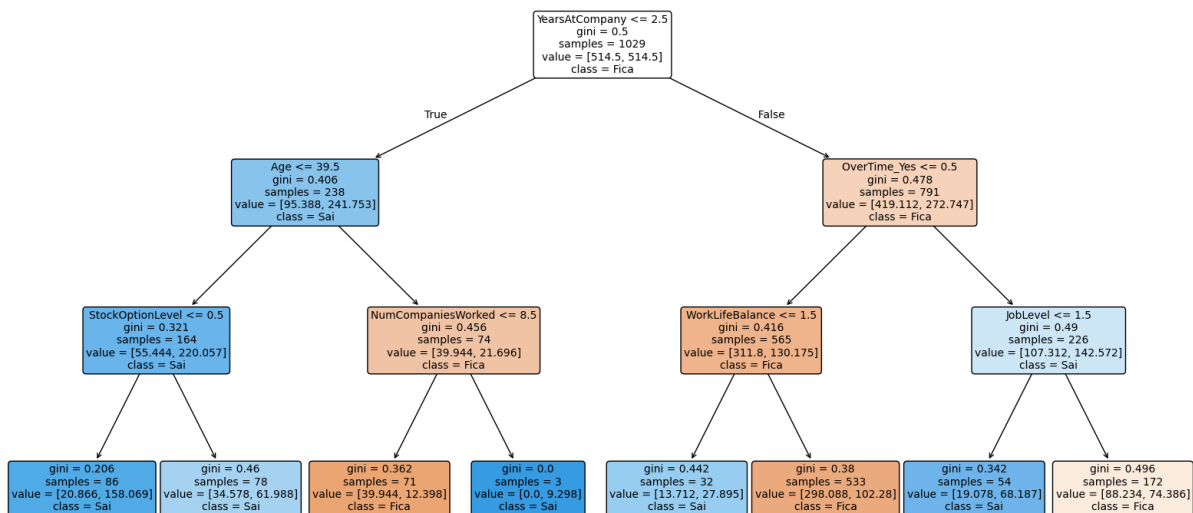
A base possui 1.470 instâncias (linhas) e atributos relevantes para o ambiente corporativo, como Idade, Rendimento Mensal e Horas Extras. O objetivo (variável alvo) é prever a coluna Attrition (rotatividade), mapeada como Sim = 1 (Sai da empresa) e Não = 0 (Fica na empresa).

3.1 Metodologia e Configuração do Modelo

A base de dados original possui forte desbalanceamento de classes (a maioria dos funcionários não sai da empresa). Para lidar com isso e treinar a Árvore de Decisão (CART via scikit-learn), a seguinte metodologia foi aplicada:

- **Pré-processamento:** Variáveis categóricas (como OverTime) foram transformadas usando One-Hot Encoding. A variável alvo foi convertida para valores binários (1 e 0).
- **Divisão de Dados:** Separação de 70% para treinamento (1.029 exemplos) e 30% para teste (441 exemplos).
- **Hiperparâmetros do Modelo:** Foi definida uma semente de aleatoriedade (random_state=42) para reprodutibilidade, profundidade máxima da árvore limitada a 3 níveis (max_depth=3) para evitar overfitting e manter a interpretabilidade, e o uso explícito do parâmetro class_weight="balanced" para compensar o desbalanceamento da base, penalizando mais os erros na classe minoritária ("Sai").

3.2 Árvore de Decisão:



- YearsAtCompany $\leq 2,5$
 - Age $\leq 39,5$
 - StockOptionLevel $\leq 0 \rightarrow$ Sai
 - StockOptionLevel $> 0 \rightarrow$ Sai
 - Age $> 39,5$
 - NumCompaniesWorked $\leq 8,5 \rightarrow$ Fica
 - NumCompaniesWorked $> 8,5 \rightarrow$ Sai
- YearsAtCompany $> 2,5$
 - OverTime = Não
 - WorkLifeBalance $\leq 1,5 \rightarrow$ Sai
 - WorkLifeBalance $> 1,5 \rightarrow$ Fica
 - OverTime = Sim
 - JobLevel $\leq 1,5 \rightarrow$ Sai
 - JobLevel $> 1,5 \rightarrow$ Fica

Podemos traduzir a árvore para as seguintes regras de negócio corporativas:

- **R1:** SE YearsAtCompany $\leq 2,5$ E Age $\leq 39,5$ ENTÃO Attrition = 1 (Sai)
- **R2:** SE YearsAtCompany $\leq 2,5$ E Age $> 39,5$ E NumCompaniesWorked $\leq 8,5$ ENTÃO Attrition = 0 (Fica)
- **R3:** SE YearsAtCompany $\leq 2,5$ E Age $> 39,5$ E NumCompaniesWorked $> 8,5$ ENTÃO Attrition = 1 (Sai)
- **R4:** SE YearsAtCompany $> 2,5$ E OverTime = Não E WorkLifeBalance $\leq 1,5$ ENTÃO Attrition = 1 (Sai)
- **R5:** SE YearsAtCompany $> 2,5$ E OverTime = Não E WorkLifeBalance $> 1,5$ ENTÃO Attrition = 0 (Fica)
- **R6:** SE YearsAtCompany $> 2,5$ E OverTime = Sim E JobLevel $\leq 1,5$ ENTÃO Attrition = 1 (Sai)
- **R7:** SE YearsAtCompany $> 2,5$ E OverTime = Sim E JobLevel $> 1,5$ ENTÃO Attrition = 0 (Fica)

Recuperando as contagens reais do treino a partir dos pesos dos nós, o risco de saída se concentra em três perfis: funcionários até 39,5 anos com até 2,5 anos de casa (R1, cerca de 43% de saída); funcionários de nível básico (JobLevel 1) com horas extras e mais de 2,5 anos de casa (R6, cerca de 41%); e funcionários sem horas extras, com mais de 2,5 anos de casa e equilíbrio trabalho-vida ruim (R4, cerca de 28%). O grupo mais estável é o de quem tem mais de 2,5 anos de casa, não faz hora extra e tem WorkLifeBalance $> 1,5$ (R5, cerca de 6% de saída). A folha de R3 tem só 3 amostras de treino e pouca confiabilidade.

3.3 Métricas (TESTE):

Acurácia: ~73,70%

Precisão: ~32,28%

Recall (Sensibilidade): ~57,75%

F1-Score: ~41,41%

Real fica: 284 TN (previsto fica) 86FP (previsto sai)

Real sai: 30 FN (previsto fica) 41 TP (previsto sai)

A acurácia é de 73,70%. Como a base é desbalanceada, ela é uma métrica enganosa, quem melhor mede o modelo são o recall, a precisão e o F1.

O modelo identifica 57,75% dos funcionários que saem (41 de 71). mas isso custa precisão: quando o modelo diz "sai", só 32,28% realmente saem (86 falsos positivos).

Para RH, o trade-off é razoável, é preferível investigar alguns funcionários que ficariam do que deixar de perceber a maioria dos que vão sair.

3.4 Saída no terminal:

MÉTRICAS DE TREINAMENTO

Recall: 63.25%

F1-Score: 50.12%

MÉTRICAS DE TESTE

Acurácia: 73.70%

Precisão: 32.28%

Recall (Sensibilidade): 57.75%

F1-Score: 41.41%

Do treino para o teste, o Recall cai de 63,25% para 57,75% e o F1-Score de 50,12% para 41,41%. A queda é moderada e, com a árvore limitada a 3 níveis, não indica overfitting forte.

MATRIZ DE CONFUSÃO (TESTE)

Verdadeiros Negativos (Real Fica, Previsto Fica) : 284

Falsos Positivos (Real Fica, Previsto Sai) : 86

Falsos Negativos (Real Sai, Previsto Fica) : 30

Verdadeiros Positivos (Real Sai, Previsto Sai) : 41

3.5 APÊNDICE:

```
import os
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix

df = pd.read_csv("WA_Fn-UseC_-HR-Employee-Attrition.csv")

df['Attrition'] = df['Attrition'].map({'Yes': 1, 'No': 0})

# separação variáveis preditoras da variável alvo
colunas_inuteis = ['EmployeeCount', 'EmployeeNumber', 'Over18',
'StandardHours']
X = df.drop(columns=['Attrition'] + [col for col in colunas_inuteis if
col in df.columns])
y = df['Attrition']

X = pd.get_dummies(X, drop_first=True)

# divisão 70 treino 30 teste
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

```

cart_hr = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    class_weight="balanced",
    random_state=42
)
cart_hr.fit(X_train, y_train)

# métricas
y_train_pred = cart_hr.predict(X_train)
recall_treino = recall_score(y_train, y_train_pred)
f1_treino = f1_score(y_train, y_train_pred)

print(" MÉTRICAS DE TREINAMENTO")
print(f"Recall: {recall_treino:.2%}")
print(f"F1-Score: {f1_treino:.2%}\n")

y_test_pred = cart_hr.predict(X_test)
acc_teste = accuracy_score(y_test, y_test_pred)
prec_teste = precision_score(y_test, y_test_pred)
recall_teste = recall_score(y_test, y_test_pred)
f1_teste = f1_score(y_test, y_test_pred)

print("MÉTRICAS DE TESTE")
print(f"Acurácia: {acc_teste:.2%}")
print(f"Precisão: {prec_teste:.2%}")
print(f"Recall (Sensibilidade): {recall_teste:.2%}")
print(f"F1-Score: {f1_teste:.2%}\n")

# matriz de Confusão
cm = confusion_matrix(y_test, y_test_pred)
tn, fp, fn, tp = cm.ravel()
print("MATRIZ DE CONFUSÃO (TESTE)")
print(f"Verdadeiros Negativos (Real Fica, Previsto Fica) : {tn}")
print(f"Falsos Positivos (Real Fica, Previsto Sai) : {fp}")
print(f"Falsos Negativos (Real Sai, Previsto Fica) : {fn}")
print(f"Verdadeiros Positivos (Real Sai, Previsto Sai) : {tp}\n")

plt.figure(figsize=(20, 10))
plot_tree(

```

```
cart_hr,  
feature_names=X.columns,  
class_names=["Fica", "Sai"],  
filled=True,  
rounded=True,  
fontsize=10,  
proportion=False  
)  
  
plt.savefig("arvore_hr_analise.png", dpi=300, bbox_inches="tight")  
print("Árvore de decisão salva como 'arvore_hr_analise.png'")
```

4. Questão 4 (<https://github.com/luctrindade/LISTA-1-DE-IA-AB1.git>):

4.1 Metodologia e Extração de Regras

Como o PRISM exige atributos categóricos e a base HR tem muitos numéricos, para resolver este problema, optou-se pela utilização do algoritmo RIPPER (Repeated Incremental Pruning to

Produce Error Reduction), implementado através da biblioteca wittgenstein em Python. O RIPPER pertence à família dos algoritmos de cobertura, que extraem regras sequencialmente e diretamente dos dados, mas com a vantagem de aplicar técnicas avançadas de poda para evitar o overfitting.

Diferente de uma árvore de decisão que cria uma estrutura hierárquica e mutuamente exclusiva, o RIPPER extrai um conjunto de regras lógicas independentes (conectadas por um "OU" lógico global). O modelo foca em encontrar as condições específicas que levam o funcionário a sair da empresa (Attrition = 1). Caso nenhuma regra seja atendida pelo perfil do funcionário, o algoritmo assume a regra por defeito (o funcionário fica, Attrition = 0).

Base de Regras SE... ENTÃO gerada:

- R1: SE YearsAtCompany ≤ 1 E MonthlyIncome $\leq 2331,2$ E OverTime = Sim ENTÃO Attrition = 1
- R2: SE StockOptionLevel = 0 E OverTime = Sim E JobLevel = 1 ENTÃO Attrition = 1
- R3: SE YearsWithCurrManager ≤ 1 E JobLevel = 1 E BusinessTravel = Travel_Frequently ENTÃO Attrition = 1
- R4: SE StockOptionLevel = 0 E JobRole = Sales Executive E WorkLifeBalance = 2 E OverTime = Sim ENTÃO Attrition = 1
- R5: SE StockOptionLevel = 0 E TotalWorkingYears ≤ 3 E DailyRate $\leq 257,2$ ENTÃO Attrition = 1
- R6: SE JobRole = Laboratory Technician E EnvironmentSatisfaction = 1 E NumCompaniesWorked = 7 ENTÃO Attrition = 1
- R7: SE JobInvolvement = 1 E EnvironmentSatisfaction = 1 ENTÃO Attrition = 1
- R8: SE NumCompaniesWorked = 5 E MonthlyIncome entre 2331,2 e 2719,2 ENTÃO Attrition = 1
- R9: SENÃO Attrition = 0 (Regra Padrão)

4.2 Métricas de Desempenho:

- Acurácia: ~84,58%
- Precisão: ~53,06%
- Recall (Sensibilidade): ~36,62%
- F1-Score: ~43,33%

Matriz de confusão:

-Real fica: 347 TN 23 FP

-Real sai: 45FN, 26TP

A alta acurácia (84,58%) é fortemente influenciada pelo desbalanceamento da base (a maioria dos funcionários fica na empresa). O modelo prioriza errar menos quando dispara um alerta (Precisão de 53,06%), o que explica o Recall de 36,62% (identificando 26 dos 71 funcionários que realmente saíram no conjunto de teste).

4.3 Saída no terminal:

Base de Regras Geradas pelo RIPPER

[[YearsAtCompany= ≤ 1.0 ^ MonthlyIncome= ≤ 2331.2 ^ OverTime=Yes] V

[StockOptionLevel=0 ^ OverTime=Yes ^ JobLevel=1] V
 [YearsWithCurrManager=<1.0 ^ JobLevel=1 ^ BusinessTravel=Travel_Frequently] V
 [StockOptionLevel=0 ^ JobRole=SalesExecutive ^ WorkLifeBalance=2 ^ OverTime=Yes] V
 [StockOptionLevel=0 ^ TotalWorkingYears=<3.0 ^ DailyRate=<257.2] V
 [JobRole=LaboratoryTechnician ^ EnvironmentSatisfaction=1 ^ NumCompaniesWorked=7] V
 [JobInvolvement=1 ^ EnvironmentSatisfaction=1] V
 [NumCompaniesWorked=5 ^ MonthlyIncome=2331.2-2719.2]]

Métricas de Desempenho

Acurácia: 0.8458

Precisão: 0.5306

Recall: 0.3662

F1-Score: 0.4333

MATRIZ DE CONFUSÃO (TESTE)

Verdadeiros Negativos (Real Fica, Previsto Fica) : 347

Falsos Positivos (Real Fica, Previsto Sai) : 23

Falsos Negativos (Real Sai, Previsto Fica) : 45

Verdadeiros Positivos (Real Sai, Previsto Sai) : 26

4.4 Comparação com a questão 3:

Diferenças de métricas:

Acurácia: 73,70% CART 84,58% RIPPER

Precisão: 32,28% CART 53,06% RIPPER

Recall: 57,75% CART 36,62% RIPPER

F1-Score 41,41% CART 43,33% RIPPER

TP: 41 de 71 CART 26 de 71 RIPPER

FP: 86 CART 23 RIPPER

Desempenho (Precisão vs. Recall): A escolha do modelo depende da estratégia do RH. A Árvore prioriza o Recall (identifica mais saídas, mas gera falsos alarmes), enquanto o RIPPER foca na Precisão (emite menos alertas, porém altamente confiáveis). O F1-Score equivalente mostra que nenhum é absolutamente superior.

Contexto da Acurácia e Configurações: A acurácia mais alta do RIPPER é enviesada pelo desbalanceamento natural da base. A Árvore teve uma acurácia menor porque foi configurada propositalmente para focar na minoria (`class_weight="balanced"`) e com profundidade podada, o que torna a comparação estruturalmente assimétrica.

Interpretabilidade: A Árvore fornece uma leitura hierárquica, curta e mutuamente exclusiva. O RIPPER gera regras independentes e sobrepostas que capturam perfis de nicho muito específicos, correndo maior risco de overfitting em pequenos grupos.

Conclusão: Apesar das abordagens diferentes, ambos os algoritmos concordam nos principais fatores de evasão (horas extras, baixo tempo de casa e nível hierárquico inicial), indicando que esses atributos são os mais associados à saída nesta base, o que merece validação com outros dados.

4.5 APÊNDICE:

```
import pandas as pd
import os
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix
import wittgenstein as lw # Biblioteca para o RIPPER

df = pd.read_csv("WA_Fn-UseC_-HR-Employee-Attrition.csv")
df['Attrition'] = df['Attrition'].map({'Yes': 1, 'No': 0})

colunas_inuteis = ['EmployeeCount', 'EmployeeNumber', 'Over18',
'StandardHours']
df = df.drop(columns=colunas_inuteis)

X = df.drop(columns=['Attrition'])
y = df['Attrition']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# treino do Algoritmo
modelo_regras = lw.RIPPER(random_state=42)
modelo_regras.fit(X_train, y_train)

# extração e Avaliação
print("Base de Regras Geradas pelo RIPPER")
modelo_regras.out_model()

y_pred = modelo_regras.predict(X_test)

print("\n Métricas de Desempenho")
print(f"Acurácia:  {accuracy_score(y_test, y_pred):.4f}")
print(f"Precisão:  {precision_score(y_test, y_pred):.4f}")
print(f"Recall:    {recall_score(y_test, y_pred):.4f}")
print(f"F1-Score:  {f1_score(y_test, y_pred):.4f}")

y_pred = [int(p) for p in y_pred]
cm = confusion_matrix(y_test, y_pred, labels=[0, 1])
tn, fp, fn, tp = cm.ravel()
```

```
print("\nMATRIZ DE CONFUSÃO (TESTE)")
print(f"Verdadeiros Negativos (Real Fica, Previsto Fica) : {tn}")
print(f"Falsos Positivos (Real Fica, Previsto Sai) : {fp}")
print(f"Falsos Negativos (Real Sai, Previsto Fica) : {fn}")
print(f"Verdadeiros Positivos (Real Sai, Previsto Sai) : {tp}")
```

5. Questão 5 (<https://github.com/lucitrindade/LISTA-1-DE-IA-AB1.git>):

Para a resolução desta questão, foi desenvolvido um Shell genérico em Python utilizando orientação a objetos. O sistema foi programado para funcionar como uma "casca vazia", ou seja, o código-fonte não contém nenhuma regra ou fato embutido (independência de domínio). Todo o conhecimento precisa ser inserido em tempo de execução ou carregado a partir de um arquivo JSON.

O código completo está disponível no repositório do GitHub e foi estruturado nos quatro módulos exigidos pelo diagrama:

- **Editor da Base de Conhecimento:** Construído para permitir a criação, edição e exclusão de regras e fatos via terminal. Também gerencia a persistência, salvando e carregando a base em formato JSON.
- **Interface:** Utiliza expressões regulares para interpretar frases mais próximas da linguagem natural, mapeando as entradas para os atributos lógicos do sistema.
- **Engenho de Inferência:** O motor implementa as três estratégias de encadeamento:
 - Para Frente (Forward): Varre a base e dispara todas as regras possíveis a partir dos fatos conhecidos, até não haver mais deduções a fazer.
 - Para Trás (Backward): Foca em um objetivo específico. Busca as regras de trás para frente e faz perguntas ao usuário apenas quando falta um fato essencial para validar uma regra.
 - Misto: Roda o modo para frente primeiro com o que já está na memória e, em seguida, inicia a busca para trás com foco no objetivo, atualizando a base a cada nova resposta.
- **Explicação (Por quê / Como):** Módulo integrado ao motor de inferência. Se o comando "por que" for acionado durante uma pergunta, o sistema exibe a pilha de execução indicando qual regra está tentando provar. Ao final, o comando "como" varre o log da trilha de inferência para mostrar a árvore de dedução do resultado.

5.1 Exemplo de execução:

5.1.1 Cadastro de Fatos:

```
sbc> editor
— EDITOR DA BASE — ('ajuda' lista comandos, 'sair' volta)
bc> regra R1: SE A = SIM E B = SIM ENTAO C = SIM
  criada: R1: SE a = SIM E b = SIM ENTÃO c = SIM
bc> regra R2: SE C = SIM ENTAO D = SIM
  criada: R2: SE c = SIM ENTÃO d = SIM
bc> regra R3: SE D = SIM E E = SIM ENTAO F = SIM
  criada: R3: SE d = SIM E e = SIM ENTÃO f = SIM
bc> fato A = SIM
```

fato: a = SIM
bc> fato B = SIM
fato: b = SIM
bc> pergunta E | O fato E é verdadeiro?
bc> listar
FATOS:
a = SIM
b = SIM
REGRAS:
R1: SE a = SIM E b = SIM ENTÃO c = SIM
R2: SE c = SIM ENTÃO d = SIM
R3: SE d = SIM E e = SIM ENTÃO f = SIM
bc> sair

5.1.2 Encadeamento para frente:

sbc> frente

? O fato E é verdadeiro?
[resposta | por que | nao sei] » nao sei

— RESULTADO —

- c = SIM (regra R1)
- d = SIM (regra R2)

sbc> fatos
a = SIM [base]
b = SIM [base]
c = SIM [R1]
d = SIM [R2]

5.1.3 Módulo de explicação:

sbc> como d
d = SIM [deduzido por R2: SE c = SIM ENTÃO d = SIM]
c = SIM [deduzido por R1: SE a = SIM E b = SIM ENTÃO c = SIM]
a = SIM [fato da base]
b = SIM [fato da base]

5.1.4 Encadeamento para trás e Por quê?:

sbc> tras f

? O fato E é verdadeiro?
[resposta | por que | nao sei] » por que
Preciso de 'e' para verificar 'e = SIM' da regra R3.
R3: SE d = SIM E e = SIM ENTÃO f = SIM
Tudo isso para chegar no objetivo final: 'f'.

? O fato E é verdadeiro?
[resposta | por que | nao sei] » sim

— RESULTADO —

> f = SIM (digite 'como f' para a justificativa)

Aqui o motor provou **d** sozinho (via R2 e R1, usando os fatos A e B) e só perguntou **e**, porque nenhuma regra o conclui.

5.1.5 Encadeamento Misto:

sbc> misto f

? O fato E é verdadeiro?

[resposta | por que | nao sei] » sim

— RESULTADO —

> f = SIM (digite 'como f' para a justificativa)

sbc> fatos

a = SIM [base]

b = SIM [base]

c = SIM [R1]

d = SIM [R2]

e = sim [usuario]

f = SIM [R3]

No misto, o motor primeiro propaga para frente (dispara R1 e R2) e depois vai para trás atrás do objetivo. Assim, a única pergunta ao usuário é ‘e’, e ao respondê-la R3 dispara automaticamente.

5.1.6 Módulo de Explicação: Trilha de Inferência:

sbc> trilha

1. Fato inicial: a = SIM

2. Fato inicial: b = SIM

3. Encadeamento MISTO

4. Regra R1 disparada => c = SIM

5. Regra R2 disparada => d = SIM

6. Encadeamento para TRÁS - objetivo: f

7. Tentando R3 para provar f

8. Usuário respondeu: e = sim

9. Regra R3 disparada => f = SIM

5.2 APÊNDICE:

```
""
SHELL DE SISTEMA BASEADO EM CONHECIMENTO

USUÁRIO -> INTERFACE -> ENGENHO DE INFERÊNCIA <-> EXPLICAÇÃO
|
BASE DE CONHECIMENTO <- EDITOR DA BASE
```

```
1. BaseConhecimento : fatos + regras SE ... ENTÃO ... (salva/carrega
JSON)
2. EditorBC          : criar / editar / excluir regras e fatos
3. Motor             : encadeamento p/ frente, p/ trás e misto
                      + explicação: POR QUÊ?, COMO?, trilha de
inferência
4. Interface         : diálogo (comandos + frases simples em
português)
```

Uso: `python questao_5.py animal.json` (exemplo)

ou `python questao_5.py` (base vazia)

```
"""
```

```
import json, re, unicodedata
from dataclasses import dataclass, field
```

```
def norm(s):
    s = unicodedata.normalize("NFD", str(s))
    return "".join(c for c in s if unicodedata.category(c) !=
"Mn").lower().strip()
```

```
def attr_id(s):
    return re.sub(r"\s+", "_", norm(s))
```

```
def igual(a, b):
    return norm(a) == norm(b)
```

```
def sim_nao(t):
    t = norm(t)
    return "sim" if t in ("s", "sim", "yes", "y") else "nao" if t in
("n", "nao", "no") else None
```

```
def tokens(s):
    parar = {"o", "a", "os", "as", "um", "uma", "de", "do", "da",
"que", "e", "em", "voce",
            "qual", "quem", "eh", "foi", "ser", "se", "me", "para",
"por"}
    return {t for t in re.findall(r"[a-z0-9]+", norm(s)) if t not in
parar}
```

```
def parse_cond(txt):
```

```

txt = txt.strip()
if "=" in txt:
    a, v = txt.split("=", 1)
    return attr_id(a), v.strip().strip("\"'")
m = re.match(r"(?i)^\n[aã]o\s+(.+)$", txt)
return (attr_id(m[1]), "nao") if m else (attr_id(txt), "sim")

def cond_str(c):
    return f"{c[0]} = {c[1]}"

@dataclass
class Regra:
    id: str
    se: list
    entao: tuple

    def __str__(self):
        return f"{self.id}: SE {' E '.join(map(cond_str, self.se))}
ENTÃO {cond_str(self.entao)}"

RE_REGRA =
re.compile(r"^s*(?:([\w.\-]+)\s*:\s*)?SE\s+(.+?)\s+ENT[AÃ]O\s+(.+?)\s*
$", re.I)

def parse_regra(txt, id_auto):
    m = RE_REGRA.match(txt)
    if not m:
        raise ValueError("sintaxe: [ID:] SE cond E cond ... ENTÃO
atributo = valor")
    return Regra(m[1] or id_auto, [parse_cond(p) for p in
re.split(r"\s+E\s+", m[2])], parse_cond(m[3]))

class BaseConhecimento:
    def __init__(self, nome="Nova base"):
        self.nome, self.fatos, self.regras, self.perguntas = nome, {},
[], {}

    def regra(self, rid):
        return next((r for r in self.regras if norm(r.id) ==
norm(rid)), None)

    def concluem(self, attr):

```

```

        return [r for r in self.regras if r.entao[0] == attr]

    def atributos(self):
        s = set(self.fatos) | {r.entao[0] for r in self.regras} | {c[0]
for r in self.regras for c in r.se}
        return sorted(s)

    def primitivos(self):
        res = []
        for r in self.regras:
            for a, _ in r.se:
                if a not in res and not self.concluem(a):
                    res.append(a)
        return res

    def objetivos(self):
        usados = {c[0] for r in self.regras for c in r.se}
        return [a for a in self.atributos() if self.concluem(a) and a
not in usados]

    def valores(self, attr):
        vals = []
        for r in self.regras:
            for a, v in r.se + [r.entao]:
                if a == attr and norm(v) not in ("sim", "nao") and v
not in vals:
                    vals.append(v)
        return vals

    def pergunta(self, attr):
        return self.perguntas.get(attr) or attr.replace("_", "
").capitalize() + "?"

    def salvar(self, caminho):
        d = {"nome": self.nome, "perguntas": self.perguntas, "fatos":
self.fatos,
            "regras": [str(r) for r in self.regras]}
        json.dump(d, open(caminho, "w", encoding="utf-8"),
ensure_ascii=False, indent=2)

    @staticmethod
    def carregar(caminho):
        d = json.load(open(caminho, encoding="utf-8"))

```

```

        bc = BaseConhecimento(d.get("nome", "Base"))
        bc.perguntas = {attr_id(k): v for k, v in d.get("perguntas",
{}).items()}
        bc.fatos = {attr_id(k): v for k, v in d.get("fatos",
{}).items()}
        for i, t in enumerate(d.get("regras", []), 1):
            bc.regras.append(parse_regra(t, f"R{i}"))
        return bc

AJUDA_EDITOR = """EDITOR DA BASE

listar                                mostra fatos e regras
regra [ID:] SE a = x E b ENTAO c = y    cria regra (ID automático se
omitido)
editar ID SE ... ENTAO ...              substitui a regra ID
excluir ID                             remove a regra
fato atributo = valor                  cria/atualiza fato inicial
excluir fato atributo                  remove fato
pergunta atributo | texto              define a pergunta feita ao usuário
salvar arq.json | sair"""

class EditorBC:
    def __init__(self, bc):
        self.bc = bc

    def executar(self, linha):
        bc = self.bc
        cmd, _, arg = linha.strip().partition(" ")
        arg = arg.strip()
        c = norm(cmd)

        if c == "listar":
            print("FATOS:", *[f"\n {a} = {v}" for a, v in
bc.fatos.items()] or ["(nenhum)"])
            print("REGRAS:", *[f"\n {r}" for r in bc.regras] or
["(nenhuma)"])
            elif c == "regra" or RE_REGRA.match(linha):
                r = parse_regra(arg if c == "regra" else linha,
f"R{len(bc.regras) + 1}")
                if bc.regra(r.id):
                    raise ValueError(f"já existe a regra {r.id}")
                bc.regras.append(r)
                print(" criada:", r)

```

```

elif c == "editar":
    rid, _, txt = arg.partition(" ")
    antiga = bc.regra(rid)
    if not antiga:
        raise ValueError(f"regra {rid} não existe")
    nova = parse_regra(txt, antiga.id)
    nova.id = antiga.id
    bc.regras[bc.regras.index(antiga)] = nova
    print("  atualizada:", nova)
elif c == "excluir":
    if norm(arg).startswith("fato "):
        if bc.fatos.pop(attr_id(arg[5:]), None) is None:
            raise ValueError("fato não existe")
    else:
        r = bc.regra(arg)
        if not r:
            raise ValueError(f"regra {arg} não existe")
        bc.regras.remove(r)
    print("  excluído.")
elif c == "fato":
    a, v = parse_cond(arg)
    bc.fatos[a] = v
    print(f"  fato: {a} = {v}")
elif c == "pergunta":
    a, _, txt = arg.partition("|")
    bc.perguntas[attr_id(a)] = txt.strip()
elif c == "salvar":
    bc.salvar(arg or "base.json")
    print("  salvo.")
elif c == "ajuda":
    print(AJUDA_EDITOR)
elif linha.strip():
    raise ValueError("comando desconhecido (digite 'ajuda')")

def loop(self):
    print("— EDITOR DA BASE — ('ajuda' lista comandos, 'sair'
volta)")
    while True:
        try:
            linha = input("bc> ")
        except EOFError:
            return
        if norm(linha) == "sair":

```

```

        return
    try:
        self.executar(linha)
    except ValueError as e:
        print(" erro:", e)

# inferência
@dataclass
class Fato:
    valor: str
    origem: str # "base" | "usuario" | id da
regra
    premissas: list = field(default_factory=list)

class Abortar(Exception):
    pass

class Motor:
    def __init__(self, bc, io):
        self.bc, self.io = bc, io
        self.reiniciar()

    def reiniciar(self, extras=None):
        self.mem, self.trilha = {}, []
        self.feitas, self.falhas = set(), set()
        self.nao_sabe, self.em_prova = set(), set()
        self.pilha, self.moda, self.objetivo = [], None, None
        for a, v in {**self.bc.fatos, **(extras or {})} .items():
            self.mem[a] = Fato(v, "base" if a in self.bc.fatos else
"usuario")
            self.log(f"Fato inicial: {a} = {v}")

    def log(self, msg):
        self.trilha.append(f"{len(self.trilha) + 1:>3}. {msg}")

    def disparar(self, r):
        self.feitas.add(r.id)
        a, v = r.entao
        if a not in self.mem:
            self.mem[a] = Fato(v, r.id, [c[0] for c in r.sel])
            self.log(f"Regra {r.id} disparada => {a} = {v}")

```

```

def verdadeira(self, c):
    f = self.mem.get(c[0])
    return f is not None and igual(f.valor, c[1])

# encadeamento para frente
def propagar(self):
    mudou = True
    while mudou:
        mudou = False
        for r in self.bc.regras:
            if r.id not in self.feitas and r.entao[0] not in
self.mem \
                                and all(self.verdadeira(c) for c in r.se):
                self.disparar(r)
                mudou = True

def frente(self):
    self.modos = "frente"
    self.log("Encadeamento para FRENTE")
    for a in self.bc.primitivos():
        if a not in self.mem:
            r = next(r for r in self.bc.regras for c in r.se if
c[0] == a)
            self.pilha = [(r, next(c for c in r.se if c[0] == a))]
            self.perguntar(a)
    self.pilha = []
    self.propagar()

# encadeamento para trás
def tras(self, attr, valor=None):
    self.modos, self.objetivo, self.pilha = self.modos or "tras",
attr, []
    self.log(f"Encadeamento para TRÁS - objetivo: {attr}" + (f" =
{valor}" if valor else ""))
    return self.provar(attr, valor)

def provar(self, attr, valor=None):
    if attr in self.mem:
        return self.mem[attr]
    if attr in self.nao_sabe or attr in self.em_prova:
        return None

    self.em_prova.add(attr)

```

```

    regras = self.bc.concluem(attr)
    if valor:
        regras = [r for r in regras if igual(r.entao[1], valor)]

    for r in regras:
        if attr in self.mem:
            break
        if r.id in self.feitas or r.id in self.falhas:
            continue
        self.log(f"Tentando {r.id} para provar {attr}")
        if self.provar_regra(r):
            self.disparar(r)

    self.em_prova.discard(attr)
    if attr in self.mem:
        return self.mem[attr]
    return None if self.bc.concluem(attr) else self.perguntar(attr)

def provar_regra(self, r):
    for c in sorted(r.se, key=lambda c: c[0] not in self.mem):
        self.pilha.append((r, c))
        f = self.provar(c[0], c[1])
        self.pilha.pop()
        if not (f and igual(f.valor, c[1])):
            self.falhas.add(r.id)
            self.log(f"Regra {r.id} descartada: '{cond_str(c)}' nao
satisfeita")
            return False
    return True

# encadeamento misto
def misto(self, attr):
    self.modos = "misto"
    self.log("Encadeamento MISTO")
    self.propagar()
    return self.tras(attr)

def perguntar(self, attr):
    while True:
        resp = self.io.perguntar(attr)
        n = norm(resp)
        if n in ("por que", "porque", "?", "pq"):
            print(self.por_que(attr))

```

```

        elif n in ("nao sei", "ns", "desconhecido"):
            self.nao_sabe.add(attr)
            self.log(f"Usuário não sabe: {attr}")
            return None
        elif n in ("abortar", "parar"):
            raise Abortar()
        elif n:
            vals = self.bc.valores(attr)
            v = sim_nao(resp) or (vals[int(n) - 1] if n.isdigit()
and 0 < int(n) <= len(vals) else resp.strip())
            self.mem[attr] = Fato(v, "usuario")
            self.log(f"Usuário respondeu: {attr} = {v}")
            if self.modos == "misto":
                self.propagar()
            return self.mem[attr]

    def por_que(self, attr):
        if not self.pilha:
            return f" Pergunto '{attr}' porque é o objetivo da
consulta e nenhuma regra deduz isso."

        r, c = self.pilha[-1]
        if self.modos == "frente":
            linhas = [f" Coletando dados de entrada: '{attr}' é usado
na regra {r.id}."]
        else:
            linhas = [f" Preciso de '{attr}' para verificar
'{cond_str(c)}' da regra {r.id}."]

        linhas.append(f"      {r}")

        for i in range(len(self.pilha) - 2, -1, -1):
            interna, (externa, cond) = self.pilha[i + 1][0],
self.pilha[i]
            linhas.append(f" E a regra {interna.id}? Porque concluiria
{cond_str(interna.entao)}, "
                        f"necessário para '{cond_str(cond)}' da regra
{externa.id}.")

        if self.objetivo and self.modos != "frente":
            linhas.append(f" Tudo isso para chegar no objetivo final:
'{self.objetivo}'.")
        return "\n".join(linhas)

```

```

def como(self, attr, nivel=0):
    f, ind = self.mem.get(attr), " " * nivel
    if not f:
        return [f"{ind}{attr}: desconhecido"]

    if f.origem in ("base", "usuario"):
        fonte = 'fato da base' if f.origem == 'base' else
'informado pelo usuario'
        return [f"{ind}{attr} = {f.valor}    [{fonte}]"]

    linhas = [f"{ind}{attr} = {f.valor}    [deduzido por
{self.bc.regra(f.origem) or f.origem}]"]
    for p in f.premissas:
        linhas += self.como(p, nivel + 1)
    return linhas

# interface
AJUDA = """COMANDOS
    carregar arq.json | salvar arq.json | editor | regras | fatos
frente                encadeamento para frente
tras [objetivo]        encadeamento para trás    (ex.: tras animal |
tras animal = tigre)
misto [objetivo]        encadeamento misto
    como [atributo]        COMO a conclusão foi obtida        trilha
passos da inferência
    informar a = valor    informa um fato                reiniciar |
sair
Durante uma pergunta: responda, ou digite 'por que' (POR QUÊ?), 'nao
sei', 'abortar'.
Frases em português também funcionam, por exemplo:
    "qual é o animal?"    "o animal é um tigre?"    "o animal tem penas"
"o animal não voa"
    "como você concluiu o animal?"    "o que você consegue deduzir?"
"use o misto para descobrir o animal" """

class Interface:
    def __init__(self, bc=None):
        self.bc = bc or BaseConhecimento()
        self.informados = {}
        self.refazer_motor()

```

```

def refazer_motor(self):
    self.motor = Motor(self.bc, self)

def perguntar(self, attr):
    print(f"\n? {self.bc.pergunta(attr)}")
    for i, v in enumerate(self.bc.valores(attr), 1):
        print(f"    {i}) {v}")
    try:
        return input("    [resposta | por que | nao sei] » ")
    except EOFError:
        raise Abortar()

def carregar(self, caminho):
    try:
        self.bc = BaseConhecimento.carregar(caminho)
    except (OSError, ValueError, KeyError) as e:
        print("erro ao carregar:", e)
        return
    self.informados = {}
    self.refazer_motor()
    print(f"Base '{self.bc.nome}': {len(self.bc.regras)} regras,
{len(self.bc.fatos)} fatos.")

def achar_attr(self, texto, candidatos):
    tt, melhor = tokens(texto), (0, None)
    for a in candidatos:
        ref = tokens(a.replace("_", " ") + " " +
self.bc.perguntas.get(a, ""))
        if len(ref & tt) > melhor[0]:
            melhor = (len(ref & tt), a)
    return melhor[1]

def objetivo_de(self, texto):
    t = texto.strip().rstrip("?")
    if "=" in t:
        return parse_cond(t)
    a = attr_id(t) if attr_id(t) in self.bc.atributos() else \
        self.achar_attr(t, [x for x in self.bc.atributos() if
self.bc.concluem(x)])
    v = next((v for v in self.bc.valores(a) if norm(v) in norm(t)),
None) if a else None
    return a, v

```

```

def executar(self, modo, texto=""):
    if not self.bc.regras:
        return print("Base vazia: use 'editor' ou 'carregar'.")
    self.motor.reiniciar(self.informados)
    m = self.motor
    try:
        if modo == "frente":
            m.frente()
            novos = {a: f for a, f in m.mem.items() if f.origem not
in ("base", "usuario")}
            print("\n— RESULTADO —" if novos else "\nNenhuma
conclusão com os dados informados.")
            for a, f in novos.items():
                print(f"  {'*' if a in self.bc.objetivos() else
'-' } {a} = {f.valor}    (regra {f.origem})")
            return
            a, v = self.objetivo_de(texto) if texto.strip() else (None,
None)

            if texto.strip() and not a:
                return print("Objetivo não identificado. Possíveis:",
", ".join(self.bc.objetivos()))
            for alvo in [a] if a else self.bc.objetivos():
                f = m.tras(alvo, v) if modo == "tras" else
m.misto(alvo)

                if f:
                    break

                if f and (not v or igual(f.valor, v)):
                    print(f"\n— RESULTADO —\n  > {alvo} = {f.valor}
(digite 'como {alvo}' para a justificativa)")
                else:
                    print("\n— RESULTADO —\n  > Não foi possível
estabelecer o objetivo.")
            except Abortar:
                print("\nConsulta abortada.")

    def comando_como(self, texto):
        a = attr_id(texto) if attr_id(texto) in self.motor.mem else
self.achar_attr(texto, self.motor.mem)
        if not a:
            a = next((x for x in self.bc.objetivos() if x in
self.motor.mem), None)
            print("\n".join(self.motor.como(a)) if a else "Nada a explicar
ainda (faça uma consulta).")

```

```

def frase(self, linha):
    n = norm(linha)
    if n.startswith("como"):
        self.comando_como(re.sub(r"^como (voce
)?(chegou|concluiu|sabe|deduziu|obteve|descobriu)?( que| a| o)?", "",
n))

        elif "misto" in n:
            self.executar("misto",
re.sub(r".*(misto|descobrir|determinar)\s*(para)?\s*(descobrir)?", "",
n))

            elif re.search(r"para frente|deduz|conclu|inferi", n) and not
n.startswith(("qual", "quem")):
                self.executar("frente")

            elif re.match(r"^(qual|quem|que|o
que|descubra|identifique|determine|diagnostique|recomende)\b", n) \
                or n.endswith("?"):
                a, v = self.objetivo_de(n)
                if not a:
                    return print("Não identifiquei o objetivo. Possíveis:",
", ".join(self.bc.objetivos()))
                print(f"Entendi: {'verificar ' + a + ' = ' + v if v else
'descobrir ' + a}")
                self.executar("tras", f"{a} = {v}" if v else a)
            else:
                a = self.achar_attr(n, self.bc.primitivos())
                if not a:
                    return print("Não entendi. Digite 'ajuda'.")
                v = next((v for v in self.bc.valores(a) if norm(v) in n),
None) \
                    or ("nao" if re.search(r"\b(nao|nunca|sem)\b", n) else
"sim")

                self.informados[a] = v
                print(f"Entendi: {a} = {v}")

def comando(self, linha):
    cmd, _, arg = linha.strip().partition(" ")
    c, arg = norm(cmd), arg.strip()
    if c in ("sair", "exit", "quit"):
        return False
    if c == "ajuda":
        print(AJUDA)
    elif c == "carregar":

```

```

        self.carregar(arg)
    elif c == "salvar":
        self.bc.salvar(arg or "base.json")
        print("Base salva.")
    elif c == "editor":
        EditorBC(self.bc).loop()
        self.informados = {}
    elif c in ("frente", "tras", "misto"):
        self.executar(c, arg)
    elif c == "como":
        self.comando_como(arg)
    elif c == "trilha":
        print("\n".join(self.motor.trilha) or "(vazia)")
    elif c == "regras":
        print("\n".join(map(str, self.bc.regras)) or "(nenhuma)")
    elif c == "fatos":
        for a, f in self.motor.mem.items():
            print(f"  {a} = {f.valor}    [{f.origem}]")
        for a, v in self.informados.items():
            print(f"  {a} = {v}    [informado]")
    elif c == "informar":
        a, v = parse_cond(arg)
        self.informados[a] = v
    elif c == "reiniciar":
        self.informados = {}
        self.motor.reiniciar()
        print("Memória limpa.")
    elif linha.strip():
        self.frase(linha)
    return True

def run(self):
    print("=== SHELL DE SBC === ('ajuda' lista os comandos)")
    while True:
        try:
            if not self.comando(input("\nsbc> ")):
                break
        except (EOFError, KeyboardInterrupt):
            break
        except ValueError as e:
            print("erro:", e)
    print("Até logo.")

```

```
if __name__ == "__main__":  
    import sys  
    ui = Interface()  
    if len(sys.argv) > 1 and not sys.argv[1].startswith('-f'):  
        ui.carregar(sys.argv[1])  
    ui.run()
```

CÓDIGOS:

(<https://github.com/lucrindade/LISTA-1-DE-IA-AB1.git>)